

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING IN POLYMER INFORMATICS

Day 3

Unsupervised Learning and Explainable ML

Machine learning-aided network for process-property prediction of injection-molded polyamide-6 composite parts

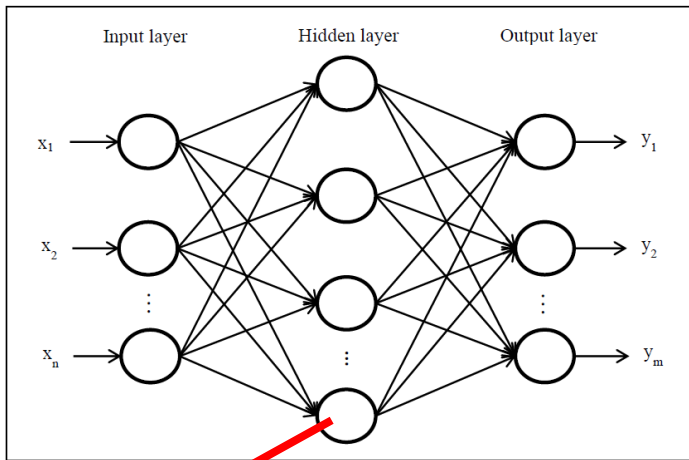
Taehwan Kim^a, Unghyeon Cho^a, Jinwoo Jang^b, Min-Kyeom Kim^a, Yongjian Fang^a, Jin Young Jung^c, Jun Yeon Hwang^c, Sang Won Lee^{a,*}, Jonghwan Suhr^{a,**}

^a Department of Mechanical Engineering, Sungkyunkwan University, Seobu-ro 2066, Jangan-gu, Suwon, 16419, Gyeonggi-do, Republic of Korea

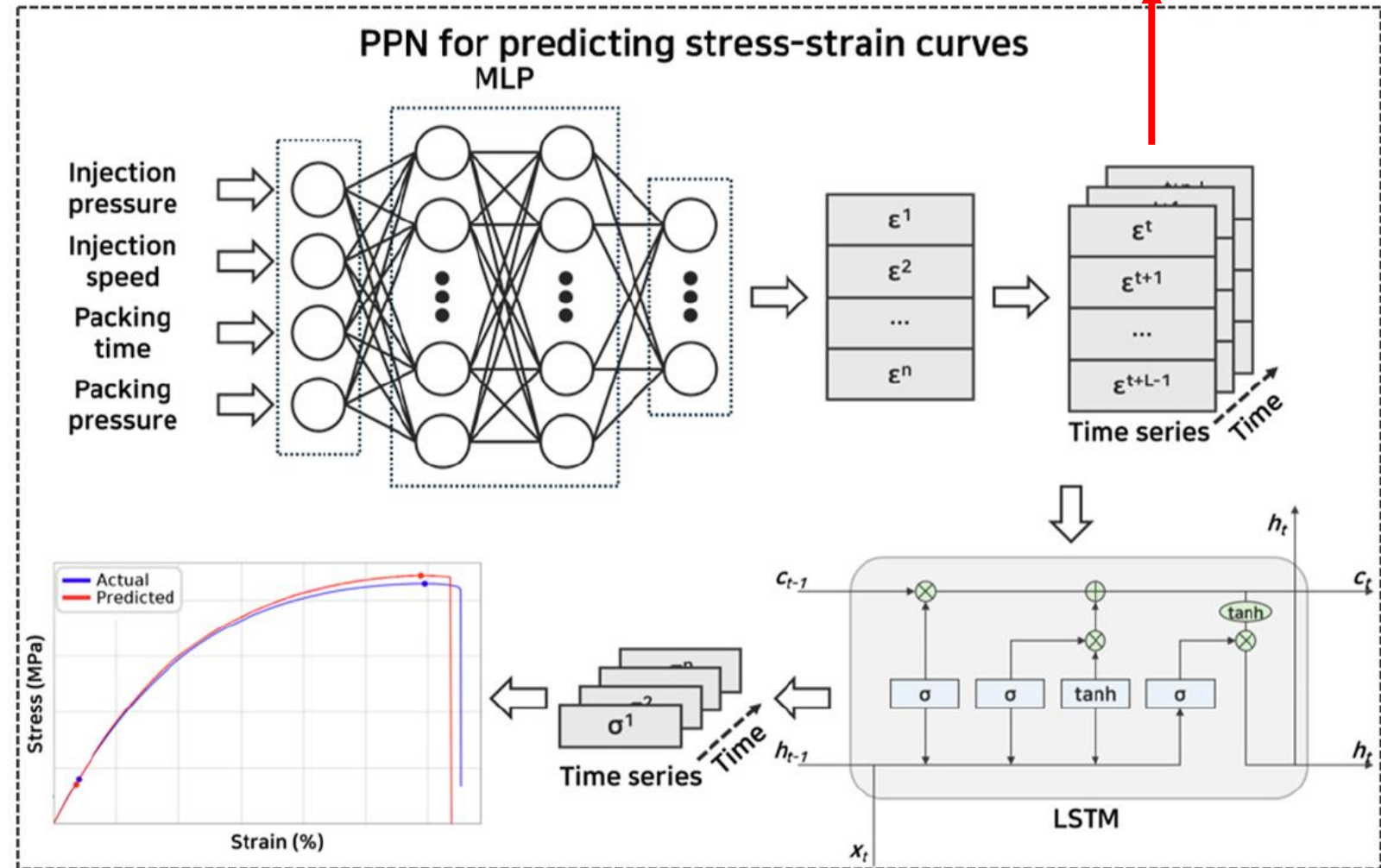
^b Department of Polymer Science, Sungkyunkwan University, Seobu-ro 2066, Jangan-gu, Suwon, 16419, Gyeonggi-do, Republic of Korea

^c Institute of Advanced Composite Materials, Korea Institute of Science and Technology (KIST), Jeonbuk, 55324, Republic of Korea

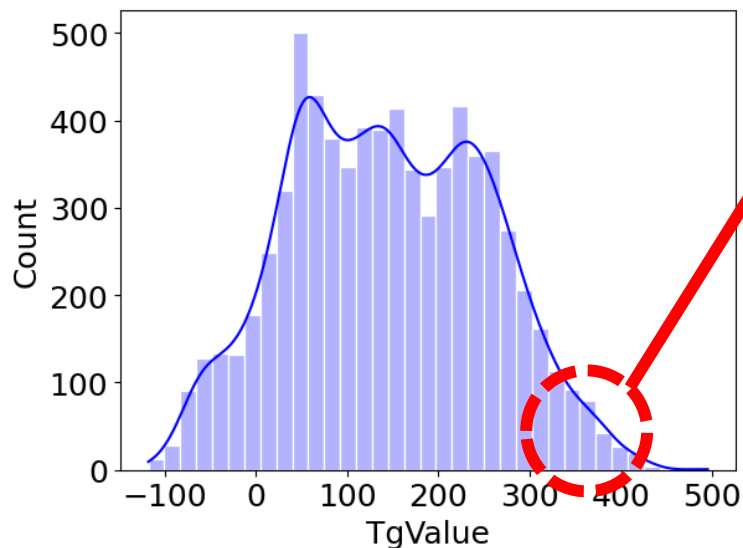
Strain points



$$a_m^{(l)} = f\left(\sum_{i=1}^n w_i^{(l)} a_i^{(l-1)} + b_i^{(l)}\right)$$

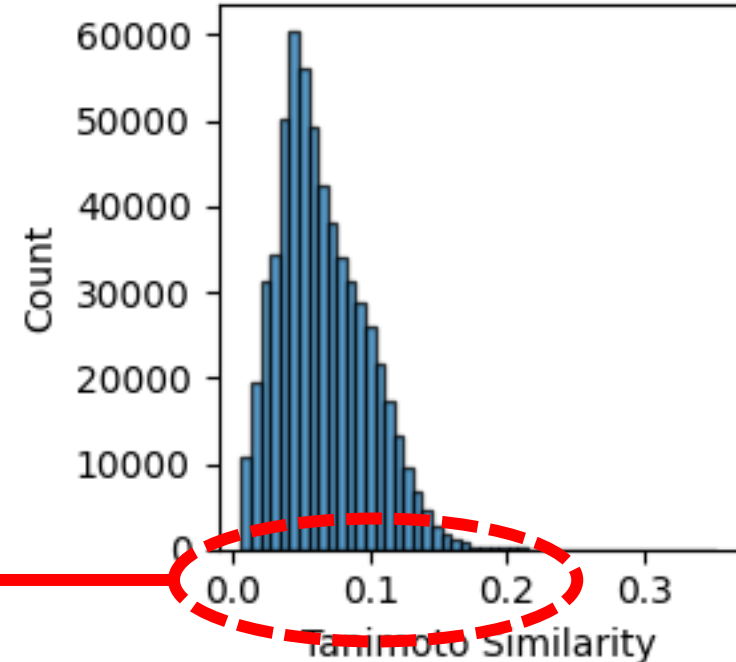


- **High prediction accuracy for promising candidates is not the key goal.**
- **A 100% success rate is not expected for screened candidates.**



High-performance targets lie in the sparse tails of the distribution

Structure gap between the training and target is unavoidable



0 (totally different) ~ 1 (Very similar)

$$T_c = \frac{c}{a + b + c}$$

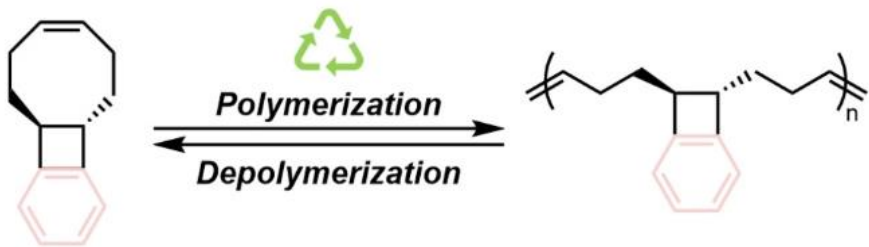
a the count of bits only “on”
in polymer A’s fingerprint

b the count of bits only “on”
in polymer B’s fingerprint

c the count of bits both “on” in
polymer A’s and B’s fingerprint

Small-Scale Testing for Candidate Library Reliability

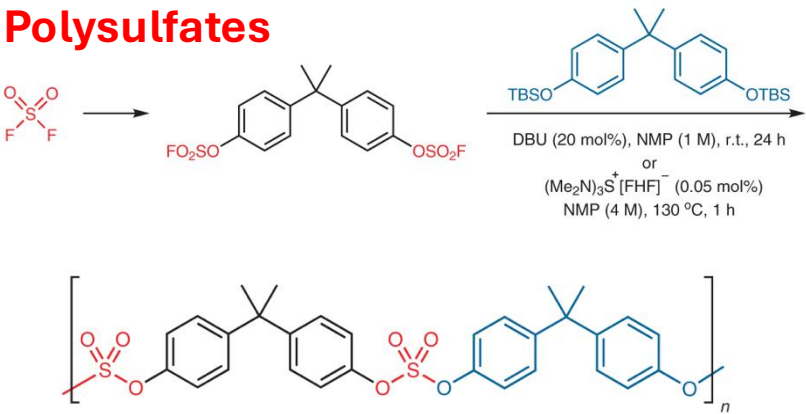
Polycyclooctenes



Junpeng Wang et al.

Model	Train R^2	Test R^2	Candidates R^2
MFF & FNN	0.99	0.82	0.84
MG & GCN	0.97	0.88	0.61

Polysulfates



Polysulfate

Yi Liu et al.

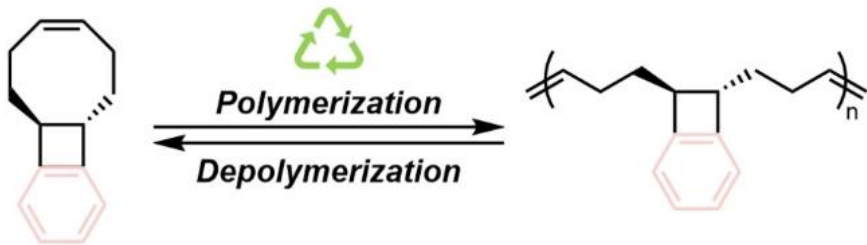
Model	Train R^2	Test R^2	Candidates R^2
MF & FNN	0.99	0.89	-2.40

- R^2 on the test set reflects model fit but does not fully capture its generalization to the target candidate library.
- The best case is to have a labeled test set within the candidate library, which allows direct evaluation of model suitability.

Evaluating Model Performance and Reliability

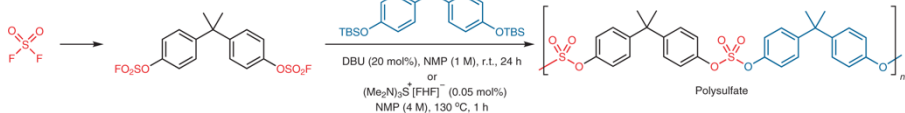
Structure similarity/ Data coverage

Polycyclooctenes



Model	Train R^2	Test R^2	Candidates R^2
MFF & FNN	0.99	0.82	0.84
MG & GCN	0.97	0.88	0.61

Polysulfates



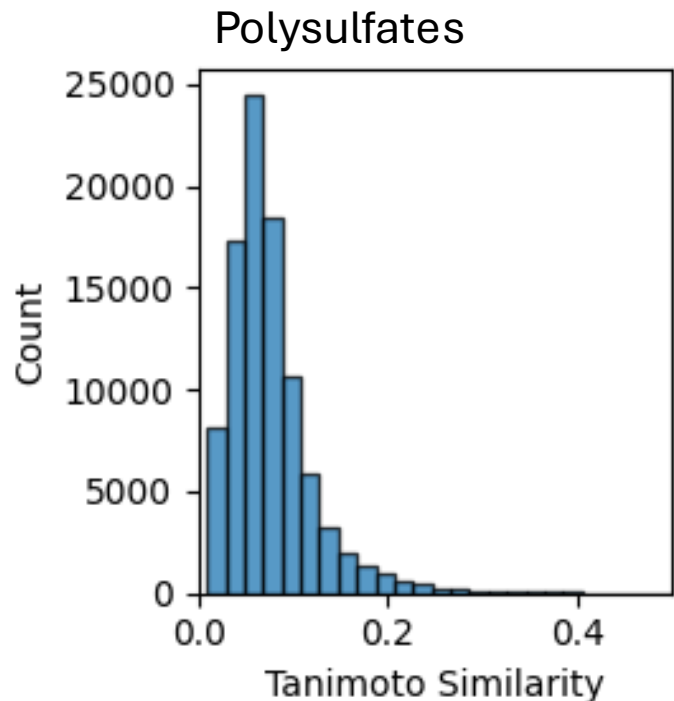
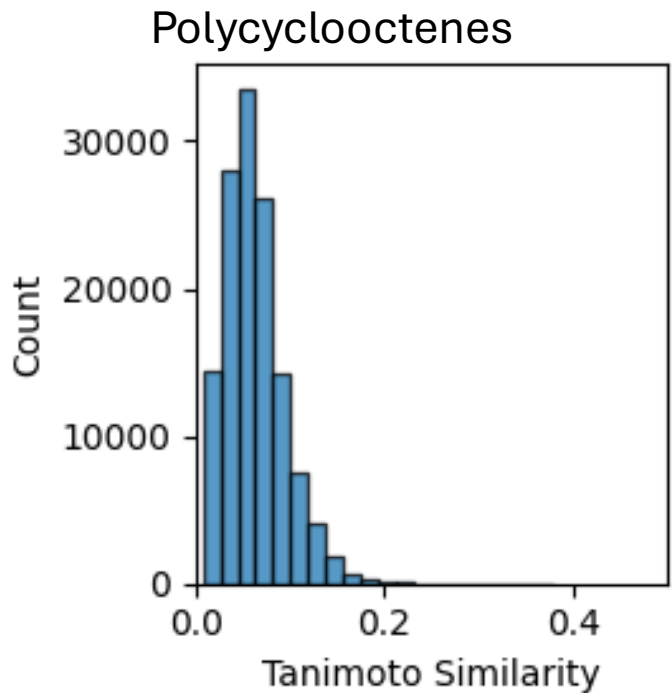
Model	Train R^2	Test R^2	Candidates R^2
MF & FNN	0.99	0.89	-2.40

$$T_c = \frac{c}{a + b + c}$$

a the count of bits only “on”
in polymer A’s fingerprint

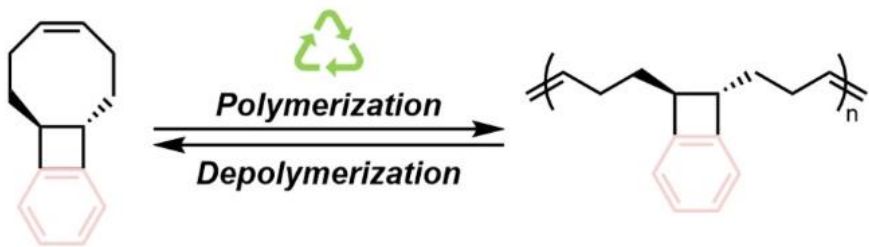
b the count of bits only “on”
in polymer B’s fingerprint

c the count of bits both “on” in
polymer A’s and B’s fingerprint

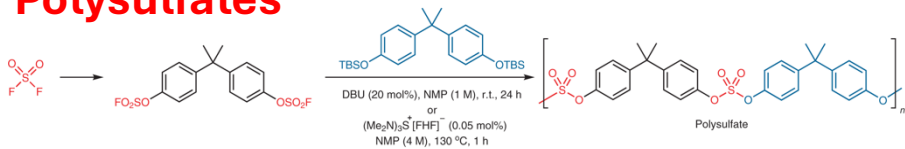


Structure similarity/ Data coverage

Polycyclooctenes

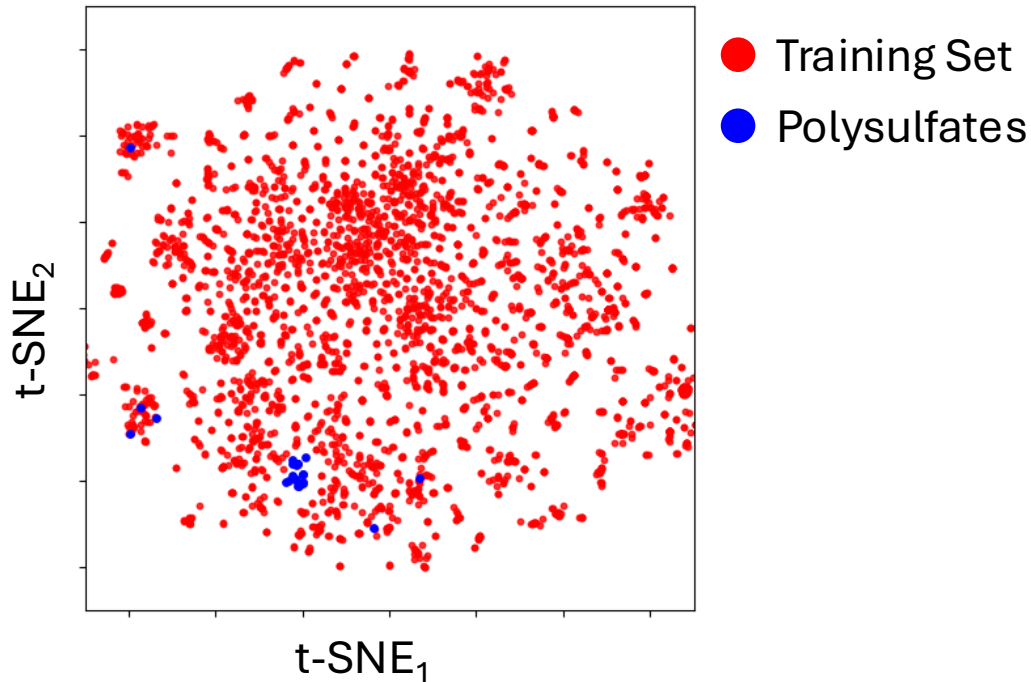
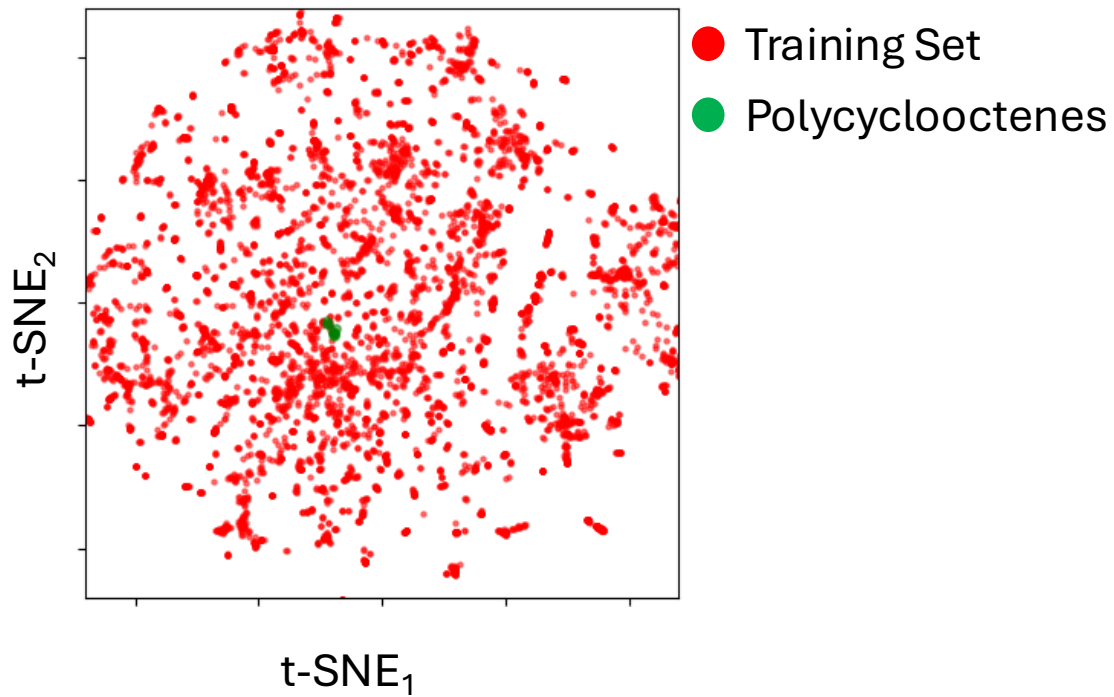


Polysulfates



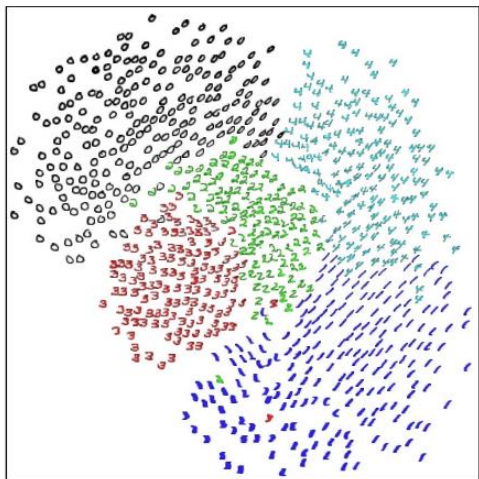
Model	Train R^2	Test R^2	Candidates R^2
MFF & FNN	0.99	0.82	0.84
MG & GCN	0.97	0.88	0.61

Model	Train R^2	Test R^2	Candidates R^2
MF & FNN	0.99	0.89	-2.40



Evaluating Model Performance and Reliability

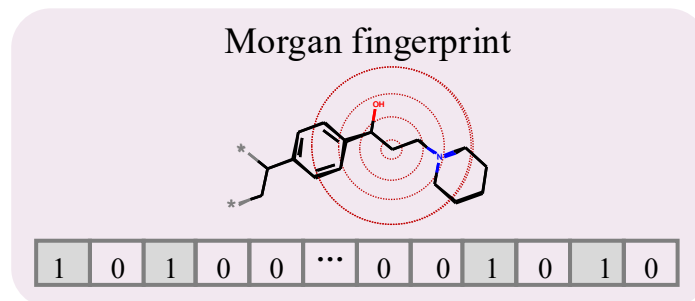
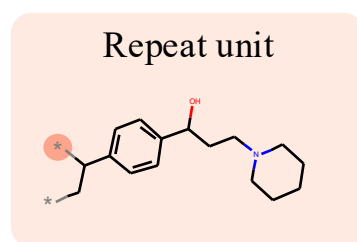
Structure similarity/ Data coverage



t-SNE plot for images of handwritten figures

t-SNE (t-distributed Stochastic Neighbor Embedding) Analysis

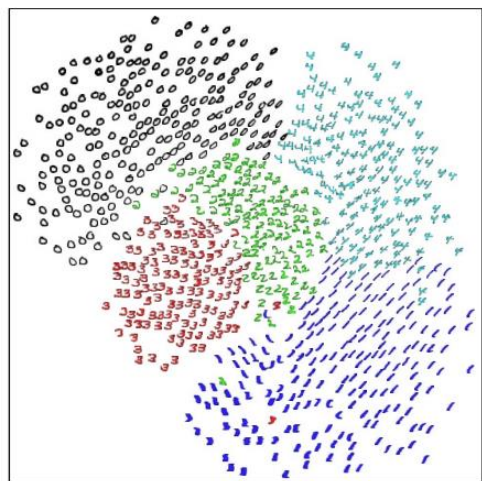
- t-SNE is a technique for visualizing high-dimensional data in 2D or 3D.
- Preserves local similarity: points that are close in high-dimensional space remain close in the low-dimensional map.
- Uses a Student t-distribution to better separate clusters.



x	y
a.aa	b.bb
c.cc	d.dd
...	...

Evaluating Model Performance and Reliability

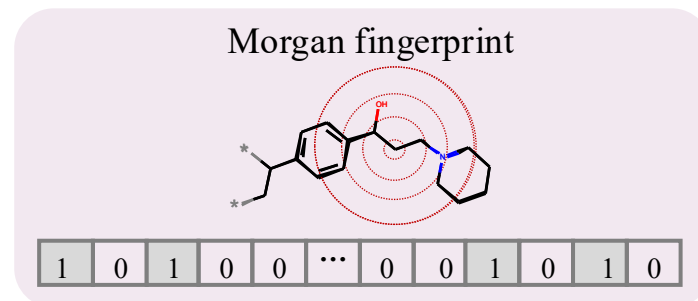
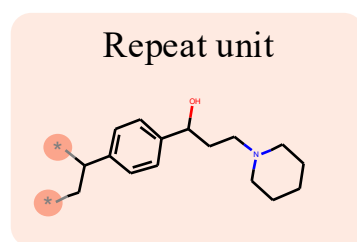
Structure similarity/ Data coverage



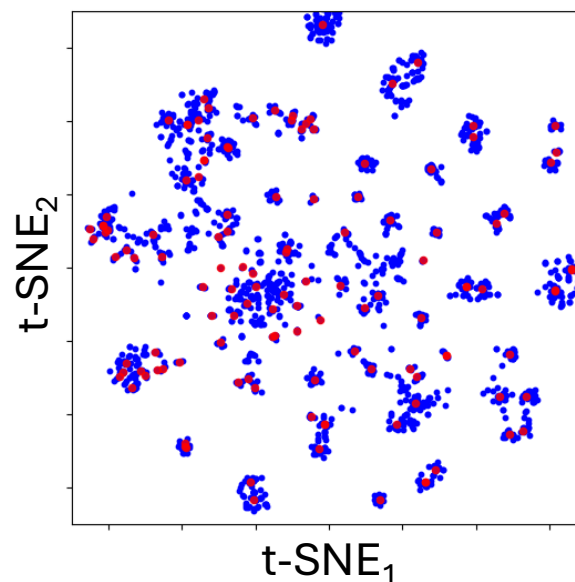
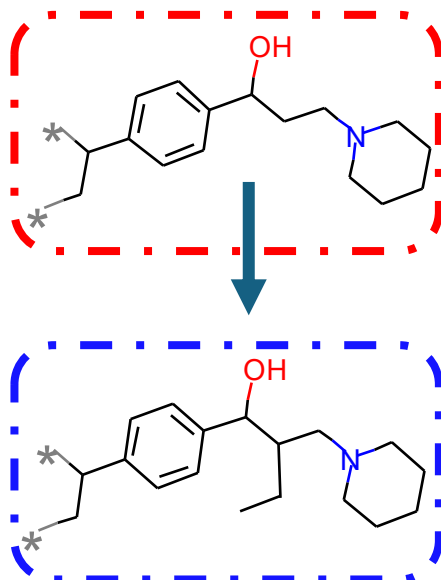
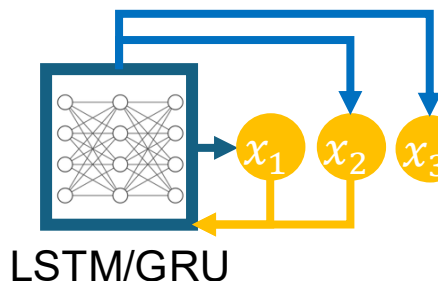
t-SNE plot for images of handwritten figures

t-SNE (t-distributed Stochastic Neighbor Embedding) Analysis

- t-SNE is a technique for visualizing high-dimensional data in 2D or 3D.
- Preserves local similarity: points that are close in high-dimensional space remain close in the low-dimensional map.
- Uses a Student t-distribution to better separate clusters.

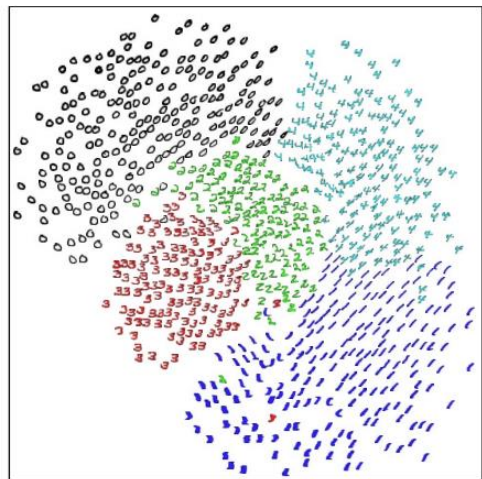


x	y
a.aa	b.bb
c.cc	d.dd
...	...



Evaluating Model Performance and Reliability

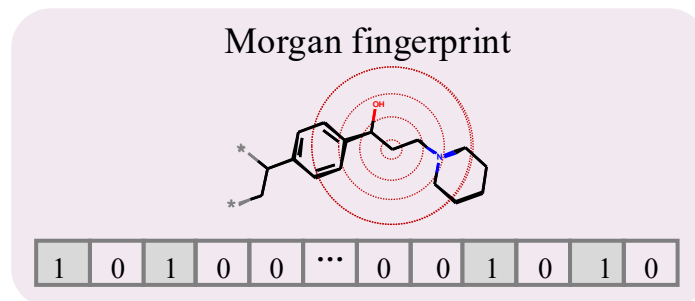
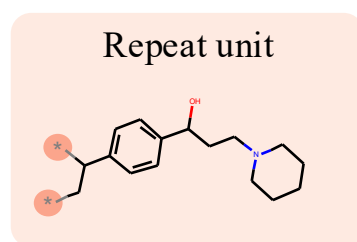
Structure similarity/ Data coverage



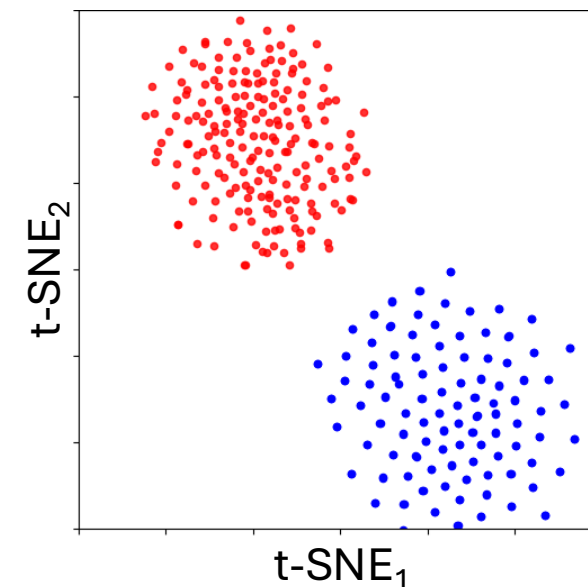
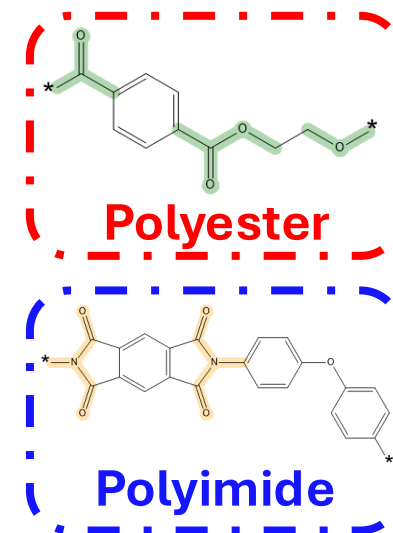
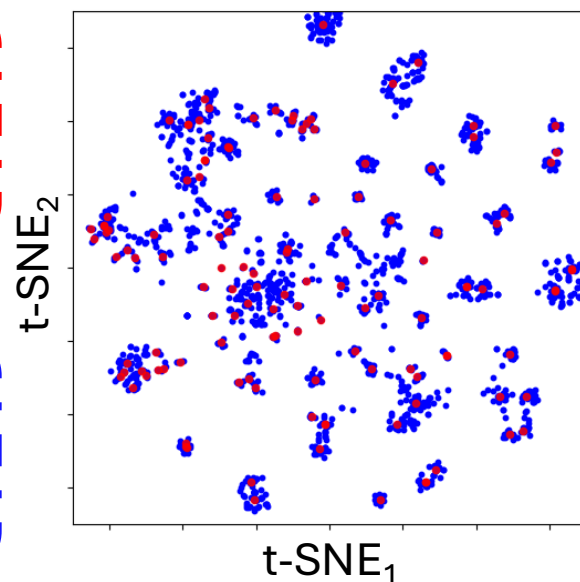
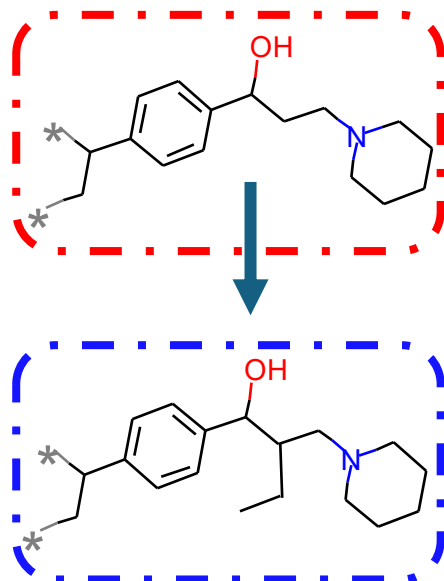
t-SNE plot for images of handwritten figures

t-SNE (t-distributed Stochastic Neighbor Embedding) Analysis

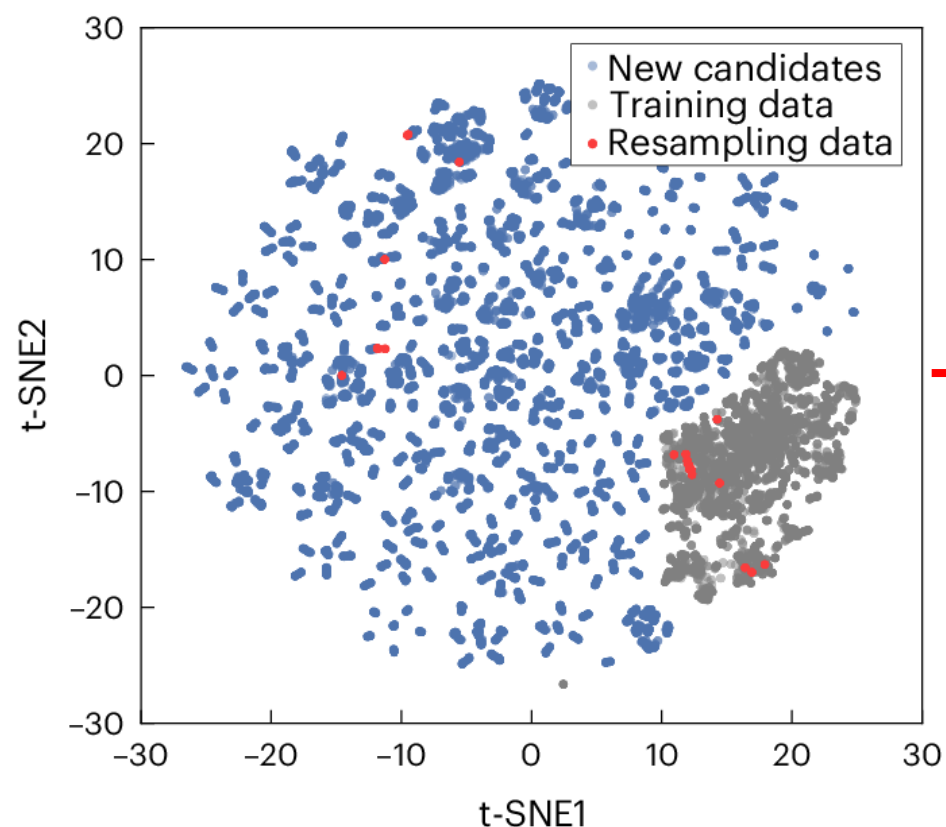
- t-SNE is a technique for visualizing high-dimensional data in 2D or 3D.
- Preserves local similarity: points that are close in high-dimensional space remain close in the low-dimensional map.
- Uses a Student t-distribution to better separate clusters.



x	y
a.aa	b.bb
c.cc	d.dd
...	...



T-Distributed Stochastic Neighbor Embedding (t-SNE)



The chemical-space distribution of a polymers (repeat units), illustrating structural similarity

Generate hypothetical polymer structures (polyether, polyimide & PBI (polybenzimidazole)) based on well-defined polymerization routes and small compounds

Use the t-SNE technique to visualize the chemical space distribution

Links your Google drive to the Colab session

```
[ ] from google.colab import drive
drive.mount('/content/drive')
```

Permit this notebook to access your Google Drive files?

This notebook is requesting access to your Google Drive files. Granting access to Google Drive will permit code executed in the notebook to modify files in your Google Drive. Make sure to review notebook code prior to allowing this access.

No thanksConnect to Google Drive

Sign in with Google

You're signing back in to Google Drive for desktop

Review Google Drive for desktop's [Privacy Policy](#) and [Terms of Service](#) to understand how Google Drive for desktop will process and protect your data.

To make changes at any time, go to your [Google Account](#).

Learn more about [Sign in with Google](#).

CancelContinue

Make sure you trust Google Drive for desktop

Review Google Drive for desktop's [Privacy Policy](#) and [Terms of Service](#) to understand how Google Drive for desktop will process and protect your data.

To make changes at any time, go to your [Google Account](#).

Learn how Google helps you [share data safely](#).

CancelContinue

Runtime duration

1m

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

Output

Links your Google drive to the Colab session

✓
0s

[2] path = `'/content/drive/MyDrive/SPE/'`  **Modify the path to the directory where your files are stored**

Check the path name is assigned correctly.

✓
0s

[3] `!echo $path`
`import os`
`os.path.isdir(path)`

 `/content/drive/MyDrive/SPE/`
`True`

 **The path is set correctly**

✓
0s

 `# This is needed so can import packages from a different path than standard libraries`
`import sys`
`sys.path.append(path)`

Check the Python packages that are available by default in the Colab environment.

Set up Python environment

Pre-installed packages 

 `!pip list`

 Package	Version

absl-py	1.4.0
accelerate	0.32.1
aiohappyeyeballs	2.4.0
aiohttp	3.10.5
aiosignal	1.3.1
alabaster	0.7.16
albucore	0.0.13
albumentations	1.4.14
altair	4.2.2
annotated-types	0.7.0
anyio	3.7.1
argon2-cffi	23.1.0
argon2-cffi-bindings	21.2.0
array_record	0.5.1
arviz	0.18.0
asn1crypto	1.5.1
astropy	6.1.2
astropy-iers-data	0.2024.8.26.0.31.57

Set up Python environment

```
!pip install rdkit  
!pip install tensorflow==2.15.0
```

Install rdkit package

Install a specific version of TensorFlow

```
!pip install shap==0.41.0
```

Install SHAP (SHapley Additive exPlanations) a game theoretic approach to explain the output of any machine learning model

```
Collecting shap==0.41.0  
  Downloading shap-0.41.0-cp310-cp310-manylinux_2_12_x86_64.manylinux2010_x86_64.whl.metadata (3.0 kB)  
Requirement already satisfied: numpy in /usr/local/lib/python3.10/dist-packages (from shap==0.41.0) (1.26.4)  
Requirement already satisfied: scipy in /usr/local/lib/python3.10/dist-packages (from shap==0.41.0) (1.13.1)  
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.10/dist-packages (from shap==0.41.0) (1.3.2)  
Requirement already satisfied: pandas in /usr/local/lib/python3.10/dist-packages (from shap==0.41.0) (2.1.4)  
Requirement already satisfied: tqdm>4.25.0 in /usr/local/lib/python3.10/dist-packages (from shap==0.41.0) (4.66.5)  
Requirement already satisfied: packaging>20.9 in /usr/local/lib/python3.10/dist-packages (from shap==0.41.0) (24.1)  
Collecting slicer==0.0.7 (from shap==0.41.0)  
  Downloading slicer-0.0.7-py3-none-any.whl.metadata (3.7 kB)  
Requirement already satisfied: numba in /usr/local/lib/python3.10/dist-packages (from shap==0.41.0) (0.60.0)  
Requirement already satisfied: cloudpickle in /usr/local/lib/python3.10/dist-packages (from shap==0.41.0) (2.2.1)  
Requirement already satisfied: llvmlite<0.44,>=0.43.0dev0 in /usr/local/lib/python3.10/dist-packages (from numba->shap==0.41.0) (0.43.0)  
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.10/dist-packages (from pandas->shap==0.41.0) (2.8.2)  
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.10/dist-packages (from pandas->shap==0.41.0) (2024.1)  
Requirement already satisfied: tzdata>=2022.1 in /usr/local/lib/python3.10/dist-packages (from pandas->shap==0.41.0) (2024.1)  
Requirement already satisfied: joblib>=1.1.1 in /usr/local/lib/python3.10/dist-packages (from scikit-learn->shap==0.41.0) (1.4.2)  
Requirement already satisfied: threadpoolctl>=2.0.0 in /usr/local/lib/python3.10/dist-packages (from scikit-learn->shap==0.41.0) (3.5.0)  
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.10/dist-packages (from python-dateutil>=2.8.2->pandas->shap==0.41.0) (1.16.0)  
Downloading shap-0.41.0-cp310-cp310-manylinux_2_12_x86_64.manylinux2010_x86_64.whl (572 kB)  
572.6/572.6 kB 7.0 MB/s eta 0:00:00  
Downloading slicer-0.0.7-py3-none-any.whl (14 kB)  
Installing collected packages: slicer, shap  
Successfully installed shap-0.41.0 slicer-0.0.7
```

Import all required packages

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

import rdkit
from rdkit import Chem
from rdkit.Chem import AllChem
from rdkit.Chem import Descriptors
from rdkit.Chem import rdMolDescriptors
from rdkit.Chem.Draw import IPythonConsole
from rdkit.Chem import Draw

from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
from sklearn.ensemble import RandomForestRegressor
from sklearn.neural_network import MLPRegressor
from sklearn.svm import SVR
from sklearn.linear_model import Lasso, LassoCV
from sklearn.model_selection import train_test_split, cross_val_predict, cross_validate
from sklearn.preprocessing import StandardScaler
from sklearn.utils import shuffle
from math import sqrt

from tensorflow.keras.models import Sequential, save_model, load_model
from tensorflow.keras.layers import Dense, Activation, Dropout
from tensorflow.keras.optimizers import Adam
from tensorflow import keras
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Conv1D, MaxPooling1D, Dense, Flatten, Activation
import time
from sklearn.metrics import r2_score

from collections import Counter
import pickle
import pandas as pd
import collections
import seaborn as sns

from rdkit import rdBase
rdBase.DisableLog('rdApp.warning')
```

Scientific computing package

Plotting

Data management

Cheminformatics toolkit

scikit-learn
Library for classical machine learning algorithms

Deep learning framework

Generate hypothetical polyimide structures

```
!pip install smipoly
```

→ Install SMiPoly (rule-based virtual library generator)

```
import numpy as np
import pandas as pd
from smipoly.smip import monc, polg
```

} → Import packages

```
#Read source file
DF = pd.read_csv(path+"202207_smip_monset.csv") #sample monomer list on GitHub
DF
```

→ Load a small compound library

	comID	MolecularFormula	MolecularWeight	SMILES	IUPACName
0	CID174	C2H6O2	62.07	C(CO)O	ethane-1,2-diol
1	CID196	C6H10O4	146.14	C(CCC(=O)O)CC(=O)O	hexanedioic acid
2	CID262	C4H10O2	90.12	CC(C(C)O)O	butane-2,3-diol
3	CID273	C5H14N2	102.18	C(CCN)CCN	pentane-1,5-diamine
4	CID371	C4H2O4	114.06	C(#CC(=O)O)C(=O)O	but-2-ynedioic acid
...	...	...	...	...	...
1078	CID91972128	C22H10N4	330.30	C=CC1=CC2=C(C=C1)C(=C(C#N)C#N)C3=CC=CC=C3C2=C(...	2-[10-(dicyanomethylidene)-3-ethenylanthracen-...
1079	CID102529357	C14H30O5Si	306.47	CO[Si](CCCCCCCCOCC1CO1)(OC)OC	trimethoxy-[8-(oxiran-2-ylmethoxy)octyl]silane
1080	CID125370311	C15H24O2Si	264.43	C=CC[Si](CCCOC(=O)C=C)(CC=C)CC=C	3-tris(prop-2-enyl)silylpropyl prop-2-enoate
1081	CID125401955	C16H26O2Si	278.46	CC(=C)C(=O)OCCC[Si](CC=C)(CC=C)CC=C	3-tris(prop-2-enyl)silylpropyl 2-methylprop-2-...
1082	CID138112368	C14H24F6N2O6S2+	494.50	CCCC[N+](C)(C)CCOC(=O)C(=C)C.C(F)(F)(F)S(=O)(=...	butyl-dimethyl-[2-(2-methylprop-2-enoyloxy)eth...

→ Small compounds library

1083 rows x 5 columns

↓
Molecular structures

Generate hypothetical polyether structures

```
# Classify monomers
DF02 = monc.moncls(df=DF, smiColn='SMILES', dsp_rsl=True)
#Generate polymers
#read source DataFrame
tpstart = time.perf_counter()
DF03 = polg.biplym(df=DF02, targ=['polyether', ], Pmode='a', dsp_rsl=True)
tpend = time.perf_counter()
tm = tpend-tpstart
print("CPU time: ", tm)
print(f"DF03 memory usage: {DF03.memory_usage(deep=True).sum() / 1024 ** 2} MB")
#extract linear polymer
DF04 = DF03[DF03['polym'].apply(lambda x:x.count('*')==2)]
DF05 = DF04.drop_duplicates(subset=['polym'])
polyether_df = DF05[['mon1', 'mon2', 'polym']].rename(columns={'polym': 'Smiles'})
polyether_df
```

Classify monomers

Set target

Generate polymers

Filter valid linear polymer SMILES

Remove duplicate SMILES

Show the results

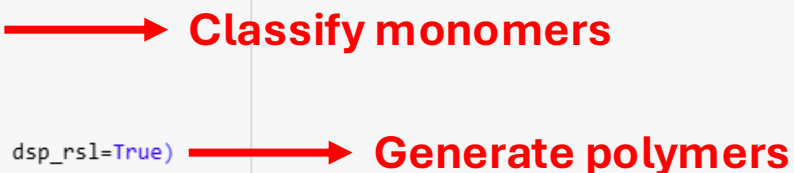
run at advanced mode
number of polymerization reactions = 739
number of generated polymers = 868
CPU time: 52.085801318999984
DF03 memory usage: 0.34459495544433594 MB

Information on results

	mon1	mon2	Smiles	
2	C1C01		*CCO*	Generated polymers
3	CC1C01		*OCC(*)C	
4	CC1C01		*CC(C)O*	
5	c1ccc(C2C02)cc1		*OCC(*)c1cccc1	
6	c1ccc(C2C02)cc1		*CC(O*)c1cccc1	
...	...	...	...	
861	O=C(c1ccc(F)cc1)c1ccc(F)cc1	OCCC1CCC(O)CC1	*OC1CCC(CC0c2ccc(C(=O)c3ccc(*)cc3)cc2)CC1	
862	O=C(c1ccc(F)cc1)c1ccc(F)cc1	OCCC1CCC(O)CC1	*OCCC1CCC(Oc2ccc(C(=O)c3ccc(*)cc3)cc2)CC1	
863	O=C(c1ccc(F)cc1)c1ccc(F)cc1	Oc1ccc(-c2ccc(O)c(Br)c2)cc1Br	*Oc1ccc(-c2ccc(Oc3ccc(C(=O)c4ccc(*)cc4)cc3)c(B...	
864	O=C(c1ccc(F)cc1)c1ccc(F)cc1	Oc1ccc(C2(c3ccc(O)c(C4CCCC4)c3)CCCC2)cc1C1CC...	*Oc1ccc(C2(c3ccc(Oc4ccc(C(=O)c5ccc(*)cc5)cc4)c...	
867	O=C(c1ccc(F)cc1)c1ccc(F)cc1	CC(C)CCC(CO)(CO)CCC(C)C	*OCC(CCC(C)C)(CCC(C)C)COc1ccc(C(=O)c2ccc(*)cc2...	

Generate hypothetical polyether structures

```
# Classify monomers
DF02 = monc.moncls(df=DF, smiColn='SMILES', dsp_rsl=True)
#Generate polymers
#read source DataFrame
tpstart = time.perf_counter()
DF03 = polg.biplym(df=DF02, targ=['polyether', ], Pmode='a', dsp_rsl=True)
tpend = time.perf_counter()
```



- Defined polymer class:
- polyolefin, polycyclic olefin and their binary copolymers
 - polyester (from lactone, hydroxy carboxylic acid, dicarboxylic acid + diol, diol + CO and cyclic carboxylic acid anhydride + epoxide)
 - polyether (from epoxide, hindered phenol, bis(halo aryl)sulfone + diol and bis(fluoro aryl)ketone + diol)
 - polyamide (from lactam, amino acid and dicarboxylic acid + diamine)
 - polyimide (bis(cyclic carboxylic acid anhydride + primary diamine)

2	-polyurethane (diisocyanate + diol)
3	
4	-polyoxazolidone (diepoxide + diisocyanate)
5	c1ccc(C2C02)cc1 *OCC(*)c1cccc1
6	c1ccc(C2C02)cc1 *CC(O*)c1cccc1
...	...
861	O=C(c1ccc(F)cc1)c1ccc(F)cc1 OCCC1CCC(O)CC1 *OC1CCC(CC0c2ccc(C(=O)c3ccc(*)cc3)cc2)CC1
862	O=C(c1ccc(F)cc1)c1ccc(F)cc1 OCCC1CCC(O)CC1 *OCCC1CCC(Oc2ccc(C(=O)c3ccc(*)cc3)cc2)CC1
863	O=C(c1ccc(F)cc1)c1ccc(F)cc1 Oc1ccc(-c2ccc(O)c(Br)c2)cc1Br *Oc1ccc(-c2ccc(Oc3ccc(C(=O)c4ccc(*)cc4)cc3)c(B...
864	O=C(c1ccc(F)cc1)c1ccc(F)cc1 Oc1ccc(C2(c3ccc(O)c(C4CCCC4)c3)CCCCC2)cc1C1CC... *Oc1ccc(C2(c3ccc(Oc4ccc(C(=O)c5ccc(*)cc5)cc4)c...
867	O=C(c1ccc(F)cc1)c1ccc(F)cc1 CC(C)CCC(CO)(CO)CCC(C)C *OCC(CCC(C)C)(CCC(C)C)COc1ccc(C(=O)c2ccc(*)cc2...

620 rows x 3 columns

Generate hypothetical polyimide structures

```
#Generate polymers
#read source DataFrame
tpstart = time.perf_counter()
DF03 = polg.biplym(df=DF02, targ=['polyimide', ], Pmode='a', dsp_rsl=True)
tpend = time.perf_counter()
tm = tpend-tpstart
print("CPU time: ", tm)
print(f"DF03 memory usage: {DF03.memory_usage(deep=True).sum() / 1024 ** 2} MB")
#extract linear polymer
DF04 = DF03[DF03['polym'].apply(lambda x:x.count('*')==2)]
DF05 = DF04.drop_duplicates(subset=['polym'])
polyimide_df = DF05[['mon1', 'mon2', 'polym']].rename(columns={'polym': 'Smiles'})
polyimide_df
```

Generate polymers

Set target

Filter valid linear polymer SMILES

Remove duplicate SMILES

Show the results

```
run at advanced mode
number of polymerization reactions = 4200
number of generated polymers = 5577
CPU time: 114.69959878999998
DF03 memory usage: 2.339837074279785 MB
```

Information on results

	mon1	mon2	Smiles
0	O=C1OC(=O)c2ccc3c4c(ccc1c24)C(=O)OC3=O	NCCCCCN	*CCCCCN1C(=O)c2ccc3c4c(ccc(c24)C1=O)C(=O)N(*)C3=O
1	O=C1OC(=O)c2ccc3c4c(ccc1c24)C(=O)OC3=O	NCCCN	*CCCN1C(=O)c2ccc3c4c(ccc(c24)C1=O)C(=O)N(*)C3=O
2	O=C1OC(=O)c2ccc3c4c(ccc1c24)C(=O)OC3=O	NCCCCN	*CCCCN1C(=O)c2ccc3c4c(ccc(c24)C1=O)C(=O)N(*)C3=O
3	O=C1OC(=O)c2ccc3c4c(ccc1c24)C(=O)OC3=O	NCCCCCCCCCN	*CCCCCCCCCN1C(=O)c2ccc3c4c(ccc(c24)C1=O)C(=O)...
4	O=C1OC(=O)c2ccc3c4c(ccc1c24)C(=O)OC3=O	Nc1ccc(S(=O)(=O)c2ccc(N)cc2)cc1	*c1ccc(S(=O)(=O)c2ccc(N3C(=O)c4ccc5c6c(ccc(c46...
...	...	...	...
5570	O=C1OC(=O)c2cc(C#Cc3ccc4c(c3)C(=O)OC4=O)ccc21	Cc1cc(C2(c3ccc(N)c(C)c3)c3ccccc3-c3ccccc32)ccc1N	*c1ccc(C2(c3ccc(N4C(=O)c5ccc(C#Cc6ccc7c(c6)C(=...
5571	O=C1OC(=O)c2cc(C#Cc3ccc4c(c3)C(=O)OC4=O)ccc21	Nc1ccc(C2(c3ccc(N)c(Cl)c3)c3ccccc3-c3ccccc32)c...	*c1ccc(C2(c3ccc(N4C(=O)c5ccc(C#Cc6ccc7c(c6)C(=...
5573	O=C1OC(=O)c2cc(C#Cc3ccc4c(c3)C(=O)OC4=O)ccc21	Nc1ccc(C2(c3ccc(N)c(F)c3)c3ccccc3-c3ccccc32)cc1F	*c1ccc(C2(c3ccc(N4C(=O)c5ccc(C#Cc6ccc7c(c6)C(=...
5574	O=C1OC(=O)c2cc(C#Cc3ccc4c(c3)C(=O)OC4=O)ccc21	Nc1ccc(OC(=O)c2ccc(C(=O)Oc3ccc(N)cc3)cc2)cc1	*c1ccc(OC(=O)c2ccc(C(=O)Oc3ccc(N4C(=O)c5ccc(C#...
5575	O=C1OC(=O)c2cc(C#Cc3ccc4c(c3)C(=O)OC4=O)ccc21	CCCCCCCCC1(CCCCCCCC)c2cc(N)ccc2-c2ccc(N)cc21	*c1ccc2c(c1)C(CCCCCCCC)(CCCCCCCC)c1cc(N3C(=O)c...

Small compounds

Generated polymers

Generate hypothetical PBI (polybenzimidazole) structures

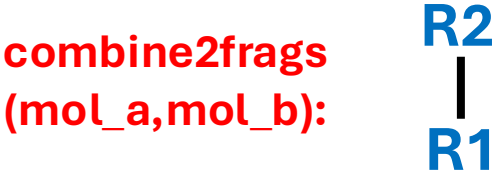
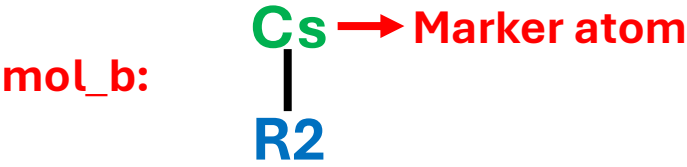
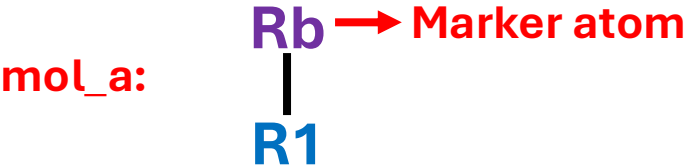
We can also use RDKit to directly generate some hypothetical polymer structures based on small molecules. Here, we will demonstrate how to generate polybenzimidazole (PBI), a polymer that is not included in the SMIPoly support list.



```
# Function to combine two molecular fragments by connecting specific marker atoms
def combine2frags(mol_a,mol_b,maker_b='Cs',maker_a='Rb'):
    # Merge the two molecules
    merged_mol = Chem.CombineMols(mol_a,mol_b)
    # Get the binding positions for the marker atoms
    bind_pos_a=get_neiid_bysymbol(merged_mol,maker_a)
    bind_pos_b=get_neiid_bysymbol(merged_mol,maker_b)
    ed_merged_mol= Chem.EditableMol(merged_mol)
    # Add a single bond between the two binding positions
    ed_merged_mol.AddBond(bind_pos_a,bind_pos_b,order=Chem.rdchem.BondType.SINGLE)
    # Remove marker atom 'maker_a'
    marker_a_idx=get_id_bysymbol(merged_mol,maker_a)
    ed_merged_mol.RemoveAtom(marker_a_idx)
    temp_mol = ed_merged_mol.GetMol()
    # Remove marker atom 'maker_b'
    marker_b_idx=get_id_bysymbol(temp_mol,maker_b)
    ed_merged_mol=Chem.EditableMol(temp_mol)
    ed_merged_mol.RemoveAtom(marker_b_idx)
    # Generate the final molecule and convert it to SMILES format
    final_mol = ed_merged_mol.GetMol()
    final_smiles = Chem.MolToSmiles(final_mol)
    return final_smiles # Return the combined molecule in SMILES format
```

Marker atom in mol_a

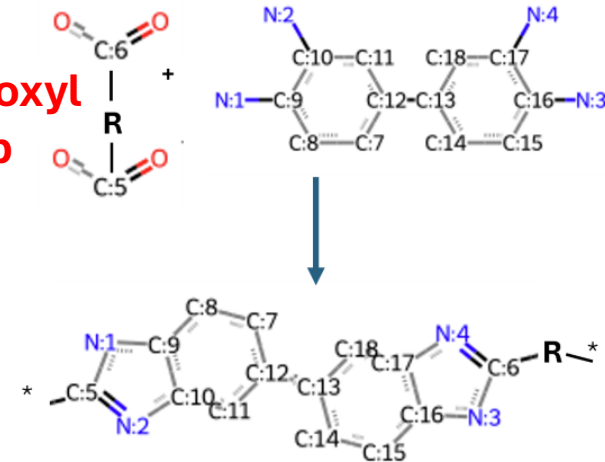
Marker atom in mol_b



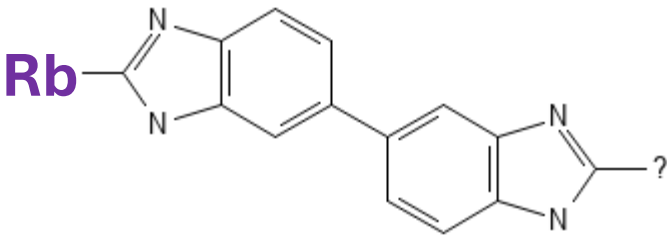
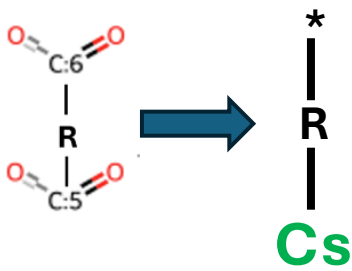
Generate hypothetical PBI (polybenzimidazole) structures

We can also use RDKit to directly generate some hypothetical polymer structures based on small molecules. Here, we will demonstrate how to generate polybenzimidazole (PBI), a polymer that is not included in the SMIPoly support list.

Carboxyl
group



Determine the reaction pathway



```
# Function to combine two molecular fragments by connecting specific marker atoms
def combine2frags(mol_a,mol_b,maker_b='Cs',maker_a='Rb'):
    # Merge the two molecules
    merged_mol = Chem.CombineMols(mol_a,mol_b)
    # Get the binding positions for the marker atoms
    bind_pos_a=get_neiid_bysymbol(merged_mol,maker_a)
    bind_pos_b=get_neiid_bysymbol(merged_mol,maker_b)
    ed_merged_mol= Chem.EditableMol(merged_mol)
    # Add a single bond between the two binding positions
    ed_merged_mol.AddBond(bind_pos_a,bind_pos_b,order=Chem.rdchem.BondType.SINGLE)
    # Remove marker atom 'maker_a'
    marker_a_idx=get_id_bysymbol(merged_mol,maker_a)
    ed_merged_mol.RemoveAtom(marker_a_idx)
    temp_mol = ed_merged_mol.GetMol()
    # Remove marker atom 'maker_b'
    marker_b_idx=get_id_bysymbol(temp_mol,maker_b)
    ed_merged_mol=Chem.EditableMol(temp_mol)
    ed_merged_mol.RemoveAtom(marker_b_idx)
    # Generate the final molecule and convert it to SMILES format
    final_mol = ed_merged_mol.GetMol()
    final_smiles = Chem.MolToSmiles(final_mol)
    return final_smiles # Return the combined molecule in SMILES format
```

Marker atom in mol_a

Marker atom in mol_b

mol_a:



mol_b:



combine2frags
(mol_a,mol_b):



Generate hypothetical PBI (polybenzimidazole) structures

```
# Import data from CSV file containing molecular information
```

```
PBI_df = pd.read_csv(path+'diCOOH.csv') → Read in small-molecule library
```

```
new_smiles_list = []
```

```
# Loop through each SMILES string in the dataframe
```

```
for smiles in PBI_df['Smiles']:
```

```
    m = Chem.MolFromSmiles(smiles) → Get the first part
```

```
    # Define patterns for modification
```

```
    patt = Chem.MolFromSmarts('C(=O)O') # Carboxylic acid group pattern → Carboxyl group
```

```
    repl1 = Chem.MolFromSmiles('[*]') # Placeholder atom for first replacement
```

```
    repl2 = Chem.MolFromSmiles('[Cs]') # Cesium atom for second replacement
```

```
    # Replace the carboxylic acid group with a placeholder atom
```

```
    rms = AllChem.ReplaceSubstructs(m, patt, repl1, replacementConnectionPoint=0) → Replace the first carboxyl group with *
```

```
    rms[0]
```

```
    # Replace the carboxylic acid group with a Cesium atom
```

```
    rms = AllChem.ReplaceSubstructs(rms[0], patt, repl2, replacementConnectionPoint=0) → Replace the second carboxyl group with marker atom
```

```
    modified_smiles = Chem.MolToSmiles(rms[0])
```

```
    modified_smiles
```

```
    # Define two molecular fragments to be combined
```

```
    mol1=Chem.MolFromSmiles('C1=CC4=C(C=C1C2=CC3=C(C=C2)N=C([N]3)[Rb])N=C([N]4)[*]') → Get the second part
```

```
    mol2=Chem.MolFromSmiles(modified_smiles)
```

```
    # Combine the two fragments and create a new SMILES string
```

```
    new_smile = combine2frags(mol1,mol2) → Combine two molecular fragments
```

```
    new_smile
```

```
    # Add the new SMILES string to the list
```

```
    new_smiles_list.append(new_smile)
```

```
# Add a new column 'mon1' with a predefined SMILES string
```

```
PBI_df['mon1'] = 'C1=CC(=C(C=C1C2=CC(=C(C=C2)N)N)N)N'
```

```
# Rename the 'Smiles' column to 'mon2'
```

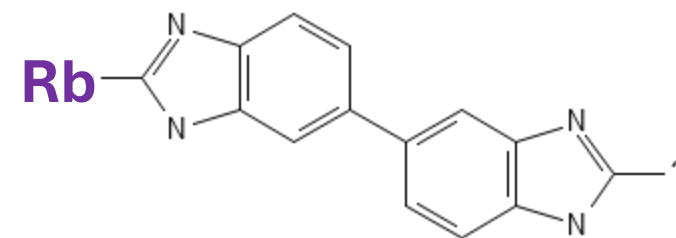
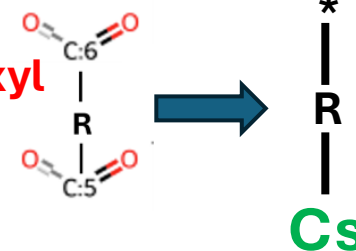
```
PBI_df.rename(columns={'Smiles': 'mon2'}, inplace=True)
```

```
# Update the 'Smiles' column with the new SMILES strings
```

```
PBI_df['Smiles'] = new_smiles_list
```

```
PBI_df # Display the updated dataframe
```

Carboxyl
group



Record generation information

Generate hypothetical PBI (polybenzimidazole) structures

Small compounds

mon2mon1

Generated polymers

Smiles

0	<chem>CC12CC1C(CC2C(O)=O)C(O)=O</chem>	<chem>C1=CC(=C(C=C1C2=CC(=C(C=C2)N)N)N)N</chem>	<chem>*C1=Nc2cc(-c3ccc4c(c3)[N]C(C3CC(*)C5(C)CC35)=N...</chem>
1	<chem>CC12CC1CC(C2C(O)=O)C(O)=O</chem>	<chem>C1=CC(=C(C=C1C2=CC(=C(C=C2)N)N)N)N</chem>	<chem>*C1=Nc2cc(-c3ccc4c(c3)[N]C(C3CC5CC5(C)C3*)=N4)...</chem>
2	<chem>CC12CC(C1)C(C2C(O)=O)C(O)=O</chem>	<chem>C1=CC(=C(C=C1C2=CC(=C(C=C2)N)N)N)N</chem>	<chem>*C1=Nc2cc(-c3ccc4c(c3)[N]C(C3C5CC(C)(C5)C3*)=N...</chem>
3	<chem>CC12C=CC(C1C2C(O)=O)C(O)=O</chem>	<chem>C1=CC(=C(C=C1C2=CC(=C(C=C2)N)N)N)N</chem>	<chem>*C1=Nc2cc(-c3ccc4c(c3)[N]C(C3C=CC5(C)C(*)C35)=...</chem>
4	<chem>CC12CC1C(C(C2)C(O)=O)C(O)=O</chem>	<chem>C1=CC(=C(C=C1C2=CC(=C(C=C2)N)N)N)N</chem>	<chem>*C1=Nc2cc(-c3ccc4c(c3)[N]C(C3C(*)CC5(C)CC35)=N...</chem>
...	...	...	...
1285	<chem>OC(=O)CCC1=CC(CC1)C(O)=O</chem>	<chem>C1=CC(=C(C=C1C2=CC(=C(C=C2)N)N)N)N</chem>	<chem>*CCC1=CC(C2=Nc3ccc(-c4ccc5c(c4)N=C(*)[N]5)cc3[...</chem>
1286	<chem>OC(=O)CCC1=CCC(C1)C(O)=O</chem>	<chem>C1=CC(=C(C=C1C2=CC(=C(C=C2)N)N)N)N</chem>	<chem>*CCC1=CCC(C2=Nc3ccc(-c4ccc5c(c4)N=C(*)[N]5)cc3...</chem>
1287	<chem>OC(=O)CCC1=CCCC1C(O)=O</chem>	<chem>C1=CC(=C(C=C1C2=CC(=C(C=C2)N)N)N)N</chem>	<chem>*CCC1=CCCC1C1=Nc2ccc(-c3ccc4c(c3)N=C(*)[N]4)cc...</chem>
1288	<chem>OC(=O)CCC1C=CCC1C(O)=O</chem>	<chem>C1=CC(=C(C=C1C2=CC(=C(C=C2)N)N)N)N</chem>	<chem>*CCC1C=CCC1C1=Nc2ccc(-c3ccc4c(c3)N=C(*)[N]4)cc...</chem>
1289	<chem>OC(=O)CCC1C(C=C)C1C(O)=O</chem>	<chem>C1=CC(=C(C=C1C2=CC(=C(C=C2)N)N)N)N</chem>	<chem>*CCC1C(C=C)C1C1=Nc2ccc(-c3ccc4c(c3)N=C(*)[N]4)...</chem>

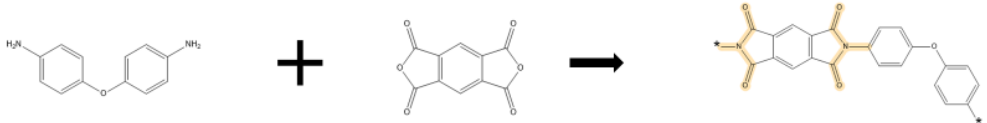
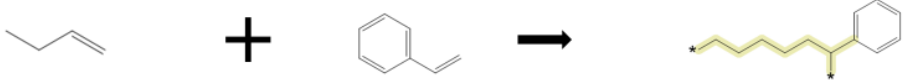
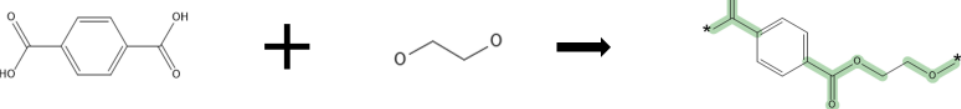
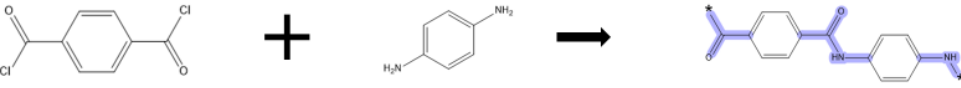
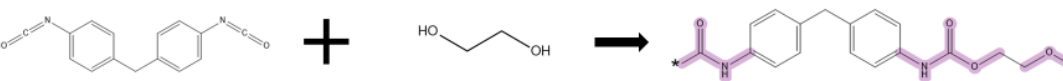
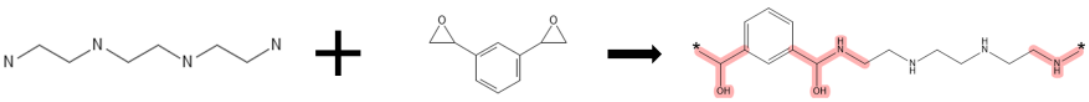
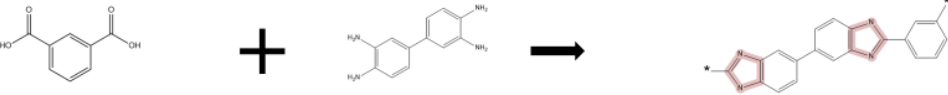
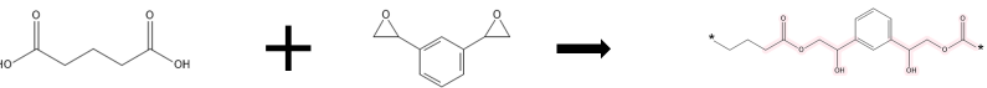
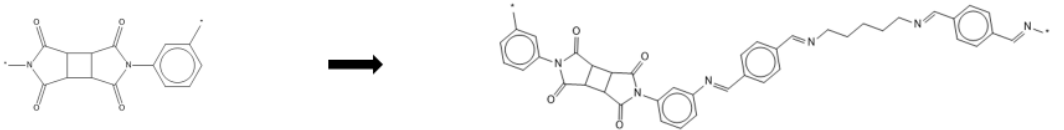
1290 rows × 3 columns

Generate hypothetical polyimide structures

DOI: [10.1039/D4DD00196F](https://doi.org/10.1039/D4DD00196F) (Paper) *Digital Discovery*, 2024, **3**, 2465-2478

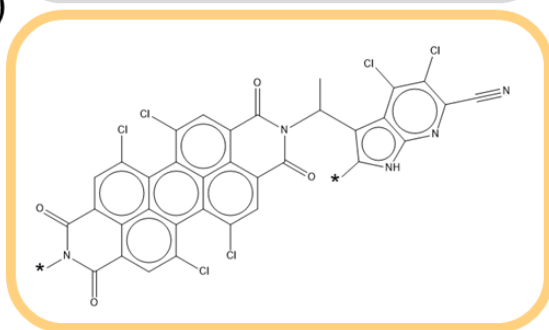
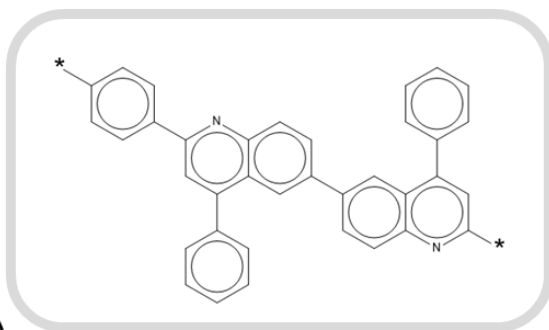
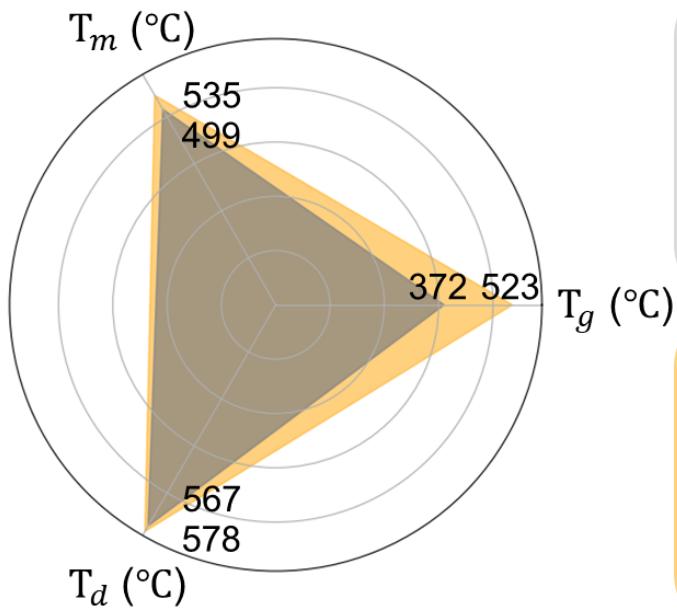
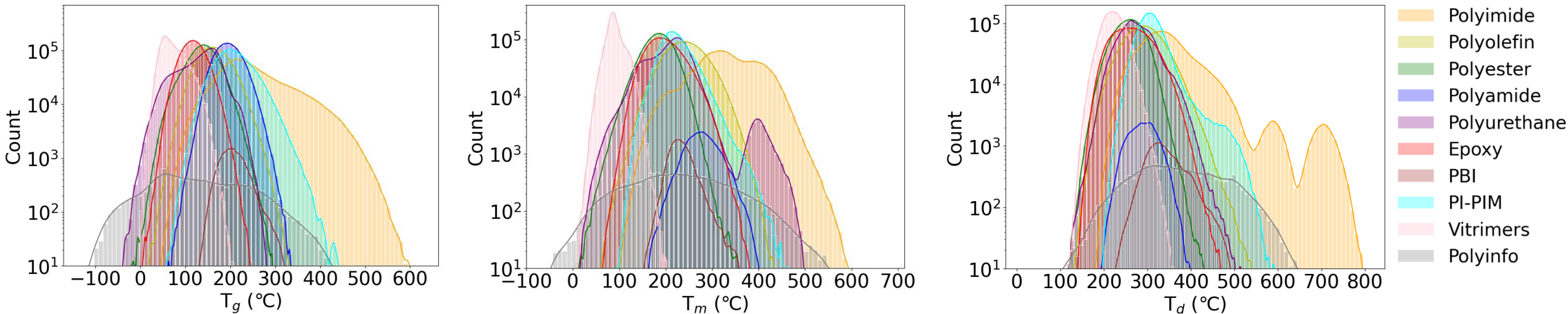
Polyuniverse: generation of a large-scale polymer library using rule-based polymerization reactions for polymer informatics†

<https://github.com/ytl0410/PolyUniverse>

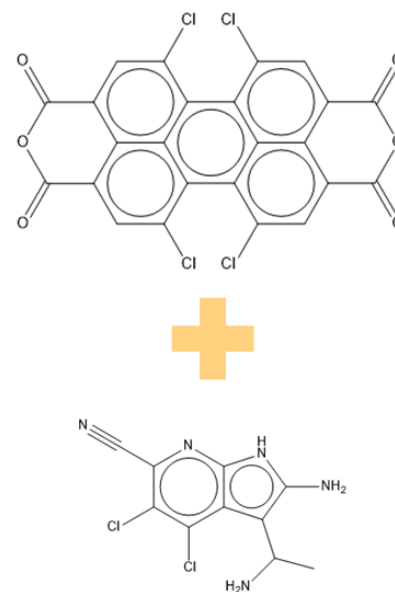
Polyimide		Monomer class	Theoretical number
Polyolefin		di/polycyclic anhydride, primary di/polyamine, polycondensation	$\sim 1.9 \times 10^{12}$
Polyester		vinyl, cyclic olefin	$\sim 1.2 \times 10^{17}$
Polyamide		lactone, hydroxy carboxylic acid, di/polyol, cyclic anhydride, epoxide	$\sim 1.8 \times 10^{13}$
Polyurethane		lactam, amino acid, di/polycarboxylic acid, di/polyamine	$\sim 1.6 \times 10^{14}$
Epoxy		di/polyisocyanate, di/polyol	$\sim 2.5 \times 10^{11}$
Polybenzimidazole (PBI)		epoxide, diamine	$\sim 3.0 \times 10^{15}$
Vitrimers		di/polycarboxylic acid, 3,3',4,4'-tetraaminodiphenyl	$\sim 5.5 \times 10^5$
Poly(imide-imine) (PI-PIM)		epoxide, di/polycarboxylic acid	$\sim 8.1 \times 10^{12}$

Generate hypothetical polyimide structures

Thermal properties

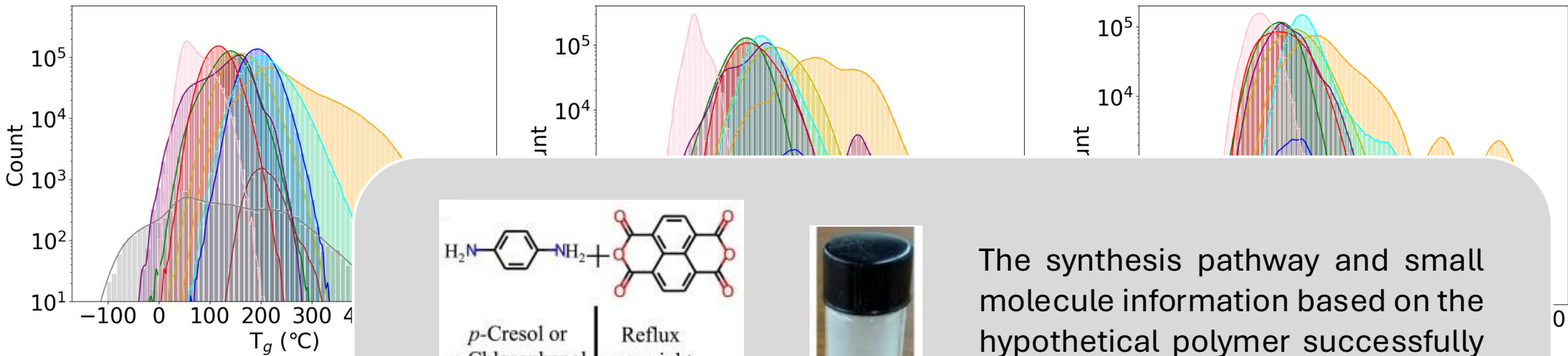


Monomers
of
Reactants

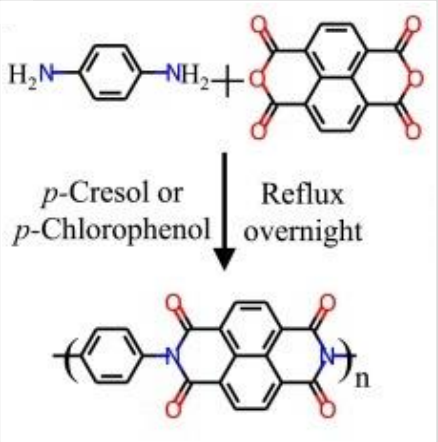


Generate hypothetical polyimide structures

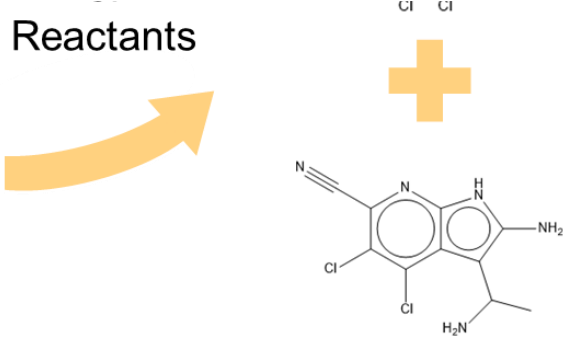
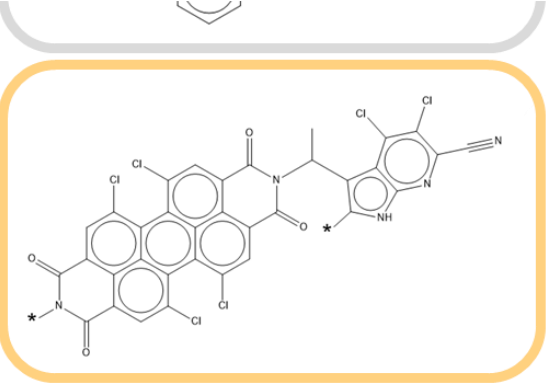
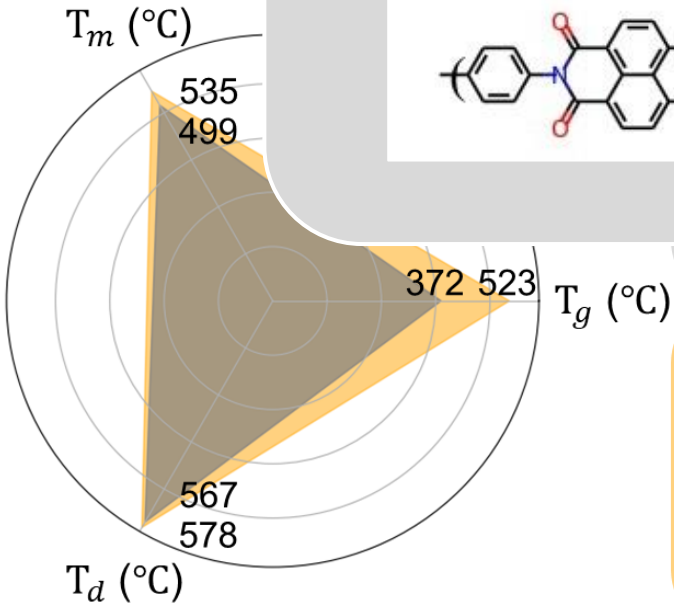
Thermal properties



- Polyimide
- Polyolefin
- Polyester
- Polyamide
- Polyurethane
- Epoxy
- PBI
- PI-PIM
- Vitrimers
- Polyinfo



The synthesis pathway and small molecule information based on the hypothetical polymer successfully led to the synthesis of #2 polyimide (low molecular weight).



T-Distributed Stochastic Neighbor Embedding (t-SNE)

```
from sklearn.manifold import TSNE
import matplotlib.pyplot as plt
```

→ Import TSNE from the scikit-learn package

```
# Combine the 'Smiles' columns from multiple DataFrames (polyether_df, polyimide_df, PBI_df, and Tg_df) into a single dataset
combined_data = pd.concat([polyether_df['Smiles'], polyimide_df['Smiles'], PBI_df['Smiles'], Tg_df['Smiles']], ignore_index=True)
```

→ Combine all the polymer structures

```
# Convert the combined SMILES strings to molecular fingerprint representations using Morgan Fingerprints (radius 3, 2048 bits)
# Drop any NaN values resulting from invalid SMILES strings
```

```
fn = pd.DataFrame(combined_data)['Smiles'].apply(Chem.MolFromSmiles).dropna().apply(lambda m: AllChem.GetMorganFingerprintAsBitVect(m, radius=3, nBits=2048))
```

→ Generate Morgan Fingerprint

```
# Convert the list of fingerprints to a NumPy array for further analysis
X_fingerprint_All_Polyinfo = np.array(fn.tolist())
```

```
# Start a timer to measure the execution time of t-SNE
time_start = time.time()
```

```
# Initialize t-SNE (t-Distributed Stochastic Neighbor Embedding) for dimensionality reduction
# The t-SNE is set to reduce the data to 2 components with a perplexity of 40 and 500 iterations
tsne = TSNE(n_components=2, verbose=1, perplexity=40, n_iter=500, random_state=100)
```

→ TSNE analysis

```
# Apply t-SNE to the molecular fingerprints to reduce dimensionality for visualization or analysis
tsne_results = tsne.fit_transform(X_fingerprint_All_Polyinfo)
```

How long the optimization will run
(More iterations allow finer convergence of the embedding)

The number of dimensions of output

Level of detail in output

Random seed

The scale at which local structure is preserved
(5~50, larger datasets → higher perplexity)

```
[t-SNE] Computing 121 nearest neighbors...
[t-SNE] Indexed 13168 samples in 0.013s...
[t-SNE] Computed neighbors for 13168 samples in 35.941s...
[t-SNE] Computed conditional probabilities for sample 1000 / 13168
[t-SNE] Computed conditional probabilities for sample 2000 / 13168
[t-SNE] Computed conditional probabilities for sample 3000 / 13168
[t-SNE] Computed conditional probabilities for sample 4000 / 13168
[t-SNE] Computed conditional probabilities for sample 5000 / 13168
[t-SNE] Computed conditional probabilities for sample 6000 / 13168
[t-SNE] Computed conditional probabilities for sample 7000 / 13168
[t-SNE] Computed conditional probabilities for sample 8000 / 13168
[t-SNE] Computed conditional probabilities for sample 9000 / 13168
[t-SNE] Computed conditional probabilities for sample 10000 / 13168
[t-SNE] Computed conditional probabilities for sample 11000 / 13168
[t-SNE] Computed conditional probabilities for sample 12000 / 13168
[t-SNE] Computed conditional probabilities for sample 13000 / 13168
[t-SNE] Computed conditional probabilities for sample 13168 / 13168
[t-SNE] Mean sigma: 2.472576
[t-SNE] KL divergence after 250 iterations with early exaggeration: 81.787163
[t-SNE] KL divergence after 500 iterations: 1.614321
```

```
tsne = TSNE(n_components=2, verbose=1, perplexity=40, n_iter=500, random_state=100)
```

T-Distributed Stochastic Neighbor Embedding (t-SNE)

Plot the results

Determine index range corresponding to each polymer

Color

Label

```
fig = plt.figure(figsize=(10, 10))
```

```
# Plot the t-SNE results for the polyether data points in red
```

```
plt.scatter(tsne_results[0:polyether_df.shape[0], 0], tsne_results[0:polyether_df.shape[0], 1], s = 15, c='red', alpha=0.3, label='polyether')
```

```
# Plot the t-SNE results for the polyimide data points in blue
```

```
plt.scatter(tsne_results[polyether_df.shape[0]:polyether_df.shape[0]+polyimide_df.shape[0], 0], tsne_results[polyether_df.shape[0]:polyether_df.shape[0]+polyimide_df.shape[0], 1], s = 15, c='blue', alpha=0.3, label='polyimide')
```

```
# Plot the t-SNE results for the PBI data points in yellow
```

```
plt.scatter(tsne_results[polyether_df.shape[0]+polyimide_df.shape[0]:polyether_df.shape[0]+polyimide_df.shape[0]+PBI_df.shape[0], 0],  
           tsne_results[polyether_df.shape[0]+polyimide_df.shape[0]:polyether_df.shape[0]+polyimide_df.shape[0]+PBI_df.shape[0], 1], s = 15, c='yellow', alpha=0.3, label='PBI')
```

```
# Plot the t-SNE results for the remaining PolyInfo data points in gray
```

```
plt.scatter(tsne_results[polyether_df.shape[0]+polyimide_df.shape[0]+PBI_df.shape[0]:, 0],  
           tsne_results[polyether_df.shape[0]+polyimide_df.shape[0]+PBI_df.shape[0]:, 1], s = 15, c='gray', alpha=0.3, label='PolyInfo')
```

```
# Set the x and y axis labels with specified font size
```

```
plt.xlabel("TSNE 1", fontsize=18)
```

```
plt.ylabel("TSNE 2", fontsize=18)
```

```
plt.xticks(fontsize=18)
```

```
plt.yticks(fontsize=18)
```

```
# Add the legend to the plot
```

```
plt.legend(fontsize=14, loc='upper right')
```

```
plt.show()
```

T-Distributed Stochastic

Plot the results

De
to

```
fig = plt.figure(figsize=(10, 10))

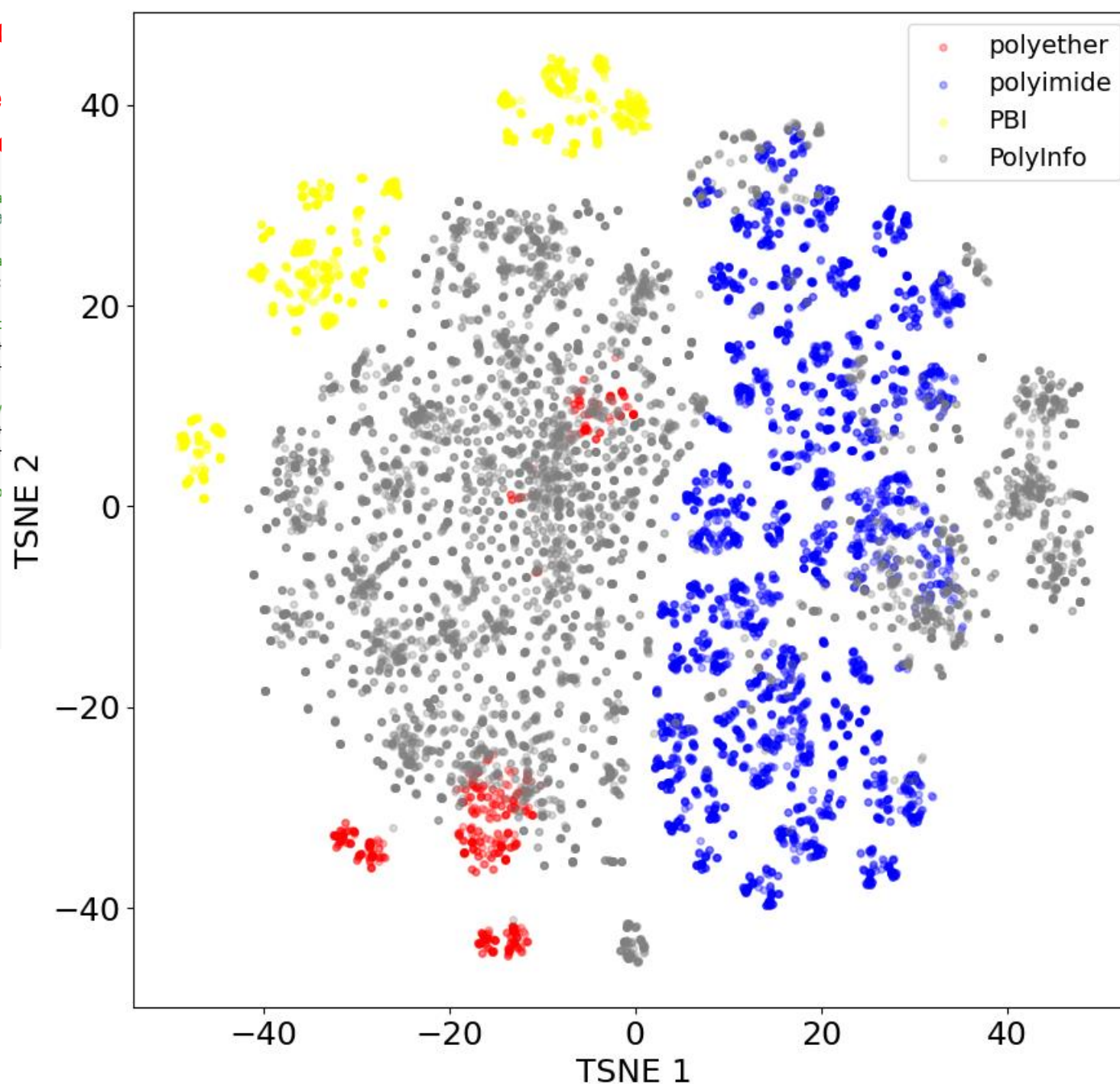
# Plot the t-SNE results for the polyether data
plt.scatter(tsne_results[0:polyether_df.shape[0],:], c='red', alpha=0.3, label='polyether')

# Plot the t-SNE results for the polyimide data
plt.scatter(tsne_results[polyether_df.shape[0]:polyimide_df.shape[0],:], c='blue', alpha=0.3, label='polyimide')

# Plot the t-SNE results for the PBI data point
plt.scatter(tsne_results[polyimide_df.shape[0]:pbi_df.shape[0],:], c='yellow', alpha=0.3, label='PBI')

# Plot the t-SNE results for the remaining PolyInfo
plt.scatter(tsne_results[pbi_df.shape[0]:tsne_results.shape[0],:], c='gray', alpha=0.3, label='PolyInfo')

# Set the x and y axis labels with specified font size
plt.xlabel("TSNE 1", fontsize=18)
plt.ylabel("TSNE 2", fontsize=18)
plt.xticks(fontsize=18)
plt.yticks(fontsize=18)
# Add the legend to the plot
plt.legend(fontsize=14, loc='upper right')
plt.show()
```



```
 = 15, c='blue', alpha=0.3, label='polyimide')
```

K-means Clustering

```
# Import the KMeans class from sklearn
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt
```

→ Import K-means from the scikit-learn package

```
# Initialize the KMeans model
# Specify the number of clusters (e.g., 4 clusters)
```

```
n_clusters = 300
```

→ Number of clusters

```
kmeans = KMeans(n_clusters=n_clusters, random_state=42)
```

→ Random seed

```
# Fit the KMeans model to the molecular fingerprint data
```

```
kmeans.fit(X_fingerprint_All_Polyinfo)
```

→ Fit the K-Means model based on fingerprint data

```
# Retrieve the cluster labels assigned to each data point
```

```
cluster_labels = kmeans.labels_
```

Determine index range corresponding
to each polymer

```
# Define the index ranges for each dataset
```

```
range_polyether = range(0, polyether_df.shape[0])
```

```
range_polyimide = range(polyether_df.shape[0], polyether_df.shape[0] + polyimide_df.shape[0])
```

```
range_PBI = range(polyether_df.shape[0] + polyimide_df.shape[0], polyether_df.shape[0] + polyimide_df.shape[0] + PBI_df.shape[0])
```

```
range_polyinfo = range(polyether_df.shape[0] + polyimide_df.shape[0] + PBI_df.shape[0], len(cluster_labels))
```

```
# Visualize the clusters
```

```
kwargs = dict(hist_kws={'alpha':.3, 'edgecolor':'white'})
```

```
sns.histplot(cluster_labels[range_polyether], kde=True, color = 'red', alpha = 0.3, edgecolor='white')
```

```
sns.histplot(cluster_labels[range_polyimide], kde=True, color = 'blue', alpha = 0.3, edgecolor='white')
```

```
sns.histplot(cluster_labels[range_PBI], kde=True, color = 'yellow', alpha = 0.3, edgecolor='white')
```

```
sns.histplot(cluster_labels[range_polyinfo], kde=True, color = 'gray', alpha = 0.3, edgecolor='white')
```

→ Plot the results

K-means Clustering

```
# Import the KMeans class from sklearn
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt
```

→ Import K-means from the scikit-learn package

```
# Initialize the KMeans model
# Specify the number of clusters (
n_clusters = 300
kmeans = KMeans(n_clusters=n_clusters)

# Fit the KMeans model to the mole
kmeans.fit(X_fingerprint_All_Polyi)

# Retrieve the cluster labels assigned
cluster_labels = kmeans.labels_
```

Determine in
to each polyr

```
# Define the index ranges for each
range_polyether = range(0, polyether_d
range_polyimide = range(polyether_d
range_PBI = range(polyether_d
range_polyinfo = range(polyether_d
```

```
# Visualize the clusters
```

```
kwargs = dict(hist_kws={'alpha':.3,
```

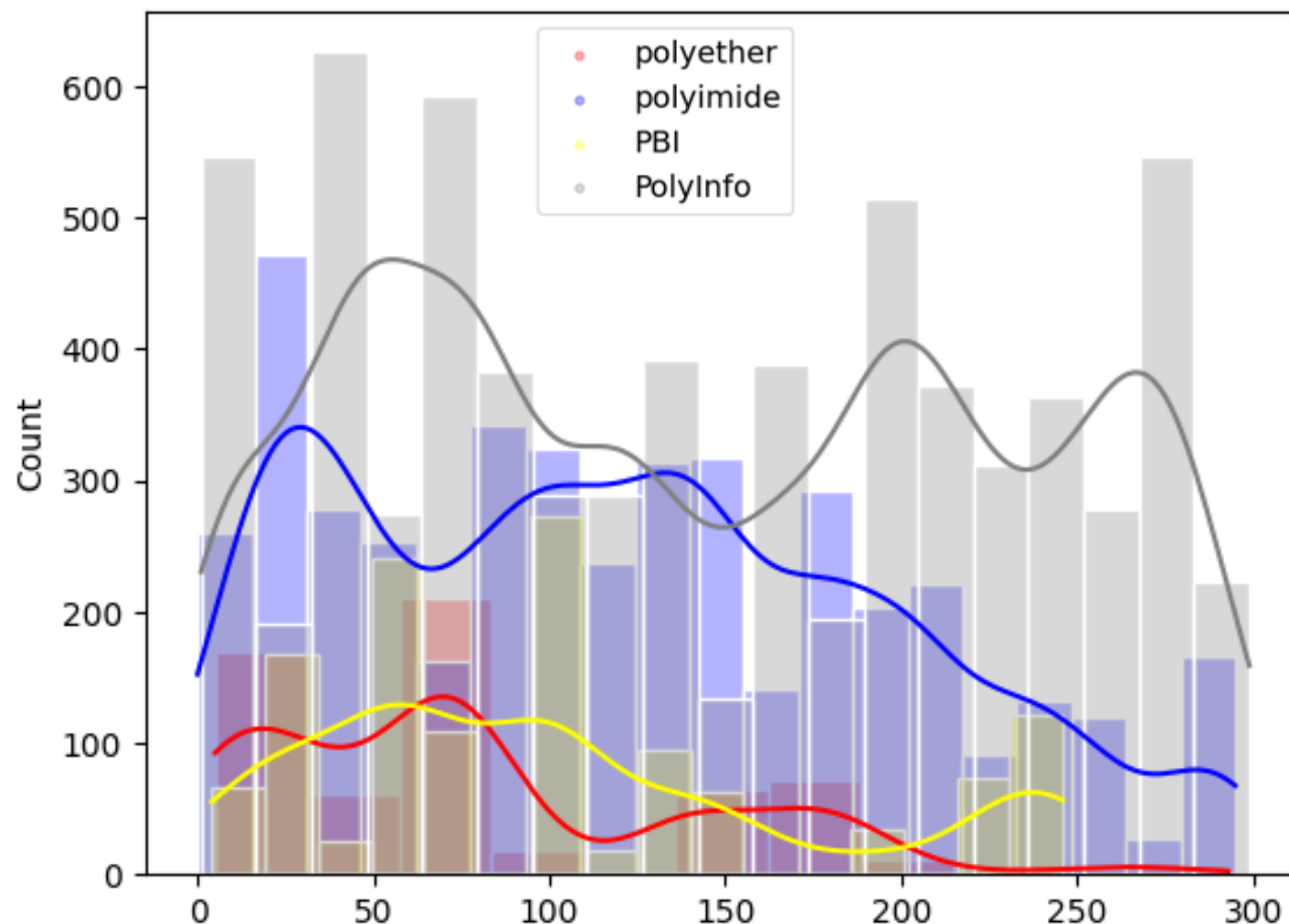
```
sns.histplot(cluster_labels[range_p
```

```
sns.histplot(cluster_labels[range_polyimide], kde=True, color = 'blue', alpha = 0.3, edgecolor='white' )
```

```
sns.histplot(cluster_labels[range_PBI], kde=True, color = 'yellow', alpha = 0.3, edgecolor='white')
```

```
sns.histplot(cluster_labels[range_polyinfo], kde=True, color = 'gray', alpha = 0.3, edgecolor='white')
```

→ Plot the results



a

SHAP (SHapley Additive exPlanations) analysis

SHAP (SHapley Additive exPlanations) is a game theoretic approach to explain the output of any machine learning model. It connects optimal credit allocation with local explanations using the classic Shapley values from game theory and their related extensions (see <https://github.com/shap/shap#citations> for details).



SHAP



SHAP (SHapley Additive exPlanations) analysis

```
[ ] import shap
    from rdkit.Chem import rdDepictor
    from rdkit.Chem.Draw import rdMolDraw2D
    import numpy as np
    np.bool_ = np.bool_
```

→ Import SHAP

```
[ ] # Randomly sample 400 rows from the DataFrame
    MFF_sampled = MFF.sample(n=400, random_state=42)
    Tg_df_sampled = Tg_df.loc[MFF_sampled.index]
    # Reset the index of the sampled DataFrame and drop the old index
    MFF_sampled = MFF_sampled.reset_index(drop=True)
    Tg_df_sampled = Tg_df_sampled.reset_index(drop=True)
    background = MFF_sampled.values
    e = shap.explainers._deep.Deep(model, background)
    shap_values = e.shap_values(background)
```

Random select 400 samples to do the analysis

Provide the model and selected samples as input

Do the SHAP analysis

```
col = 'Tg(C)'
with open("shap_values_{}.pkl".format(col), "wb") as f:
    pickle.dump(shap_values, f)

with open("shap_values_{}.pkl".format(col), "rb") as f:
    shap_values = pickle.load(f)
```

Record the SHAP information for later PCA analysis

```
shap.summary_plot(shap_values[0], MFF_sampled, feature_names=MFF_sampled.columns)
```

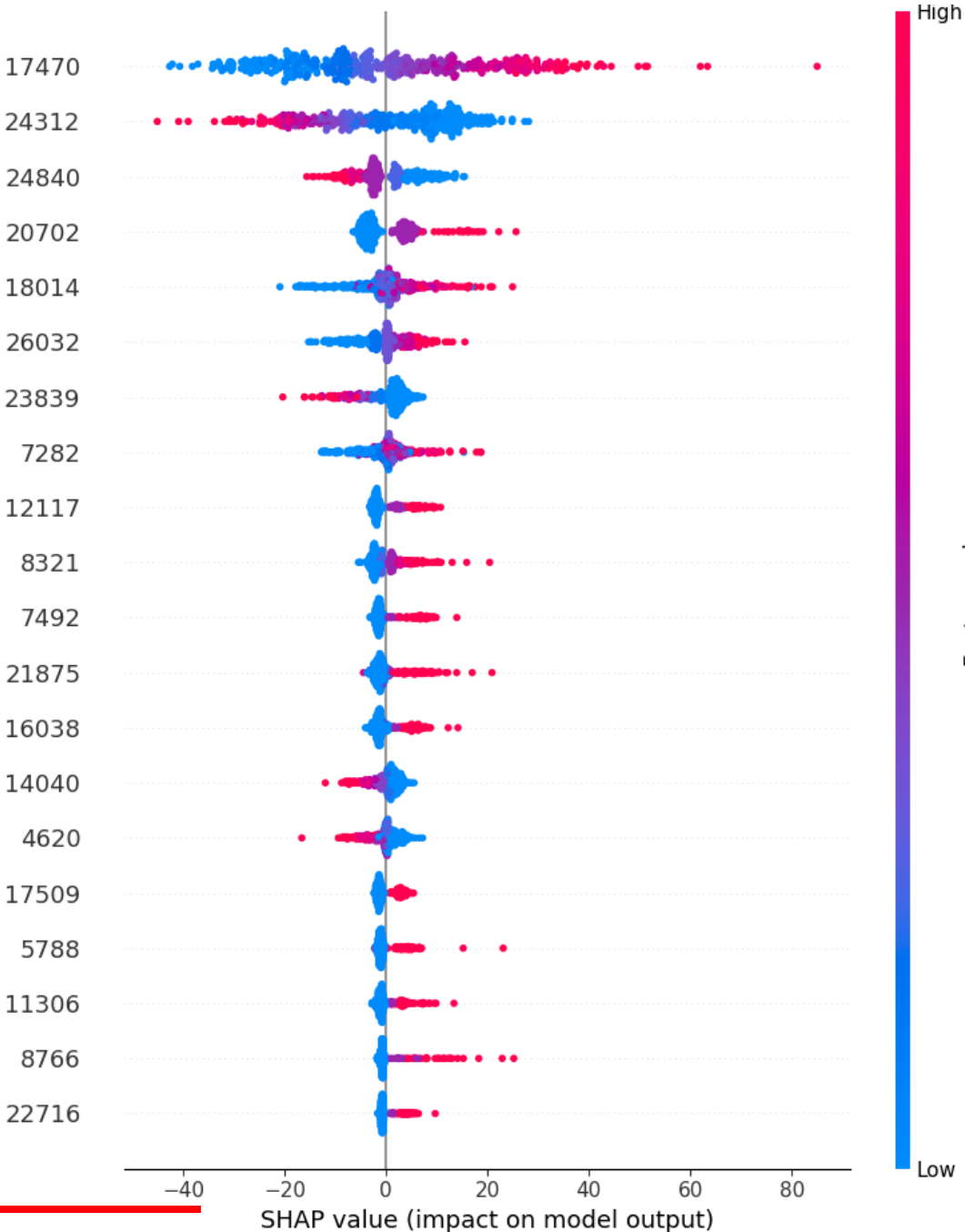
→ Plot the SHAP analysis result

SHAP (SHapley Additive exPlanations) analysis

Index of features
(Index of substructures)



17470



Feature value
(Frequency of
substructures)

Impact on the
prediction results

SHAP (SHapley Additive exPlanations) analysis

```
df_shap = shap_values[0]
df = pd.DataFrame(MFF_sampled)
#import matplotlib as plt
# Make a copy of the input data
shap_v = pd.DataFrame(df_shap)
feature_list = df.columns
shap_v.columns = feature_list
df_v = df.copy().reset_index().drop('index',axis=1)

# Determine the correlation in order to plot with different colors
corr_list = list()
for i in feature_list:
    b = np.corrcoef(shap_v[i],df_v[i])[1][0]
    corr_list.append(b)
corr_df = pd.concat([pd.Series(feature_list),pd.Series(corr_list)],axis=1).fillna(0)
# Make a data frame. Column 1 is the feature, and Column 2 is the correlation coefficient
corr_df.columns = ['Variable','Corr']
corr_df['Sign'] = np.where(corr_df['Corr']>0,'red','blue')

# Plot it
shap_abs = np.abs(shap_v)
k=pd.DataFrame(shap_abs.mean()).reset_index()
k.columns = ['Variable','SHAP_abs']
k2 = k.merge(corr_df,left_on = 'Variable',right_on='Variable',how='inner')
k2 = k2.sort_values(by='SHAP_abs',ascending = True)

with open("shap_df_{}.pkl".format(col), "wb") as f:
    pickle.dump(k2.iloc[::1], f)

k2 = k2[-20:]
colorlist = k2['Sign']
ax = k2.plot.barh(x='Variable',y='SHAP_abs',color = colorlist, figsize=(5,6),legend=False)
ax.set_xlabel("SHAP Value (Red = Positive Impact)")
```

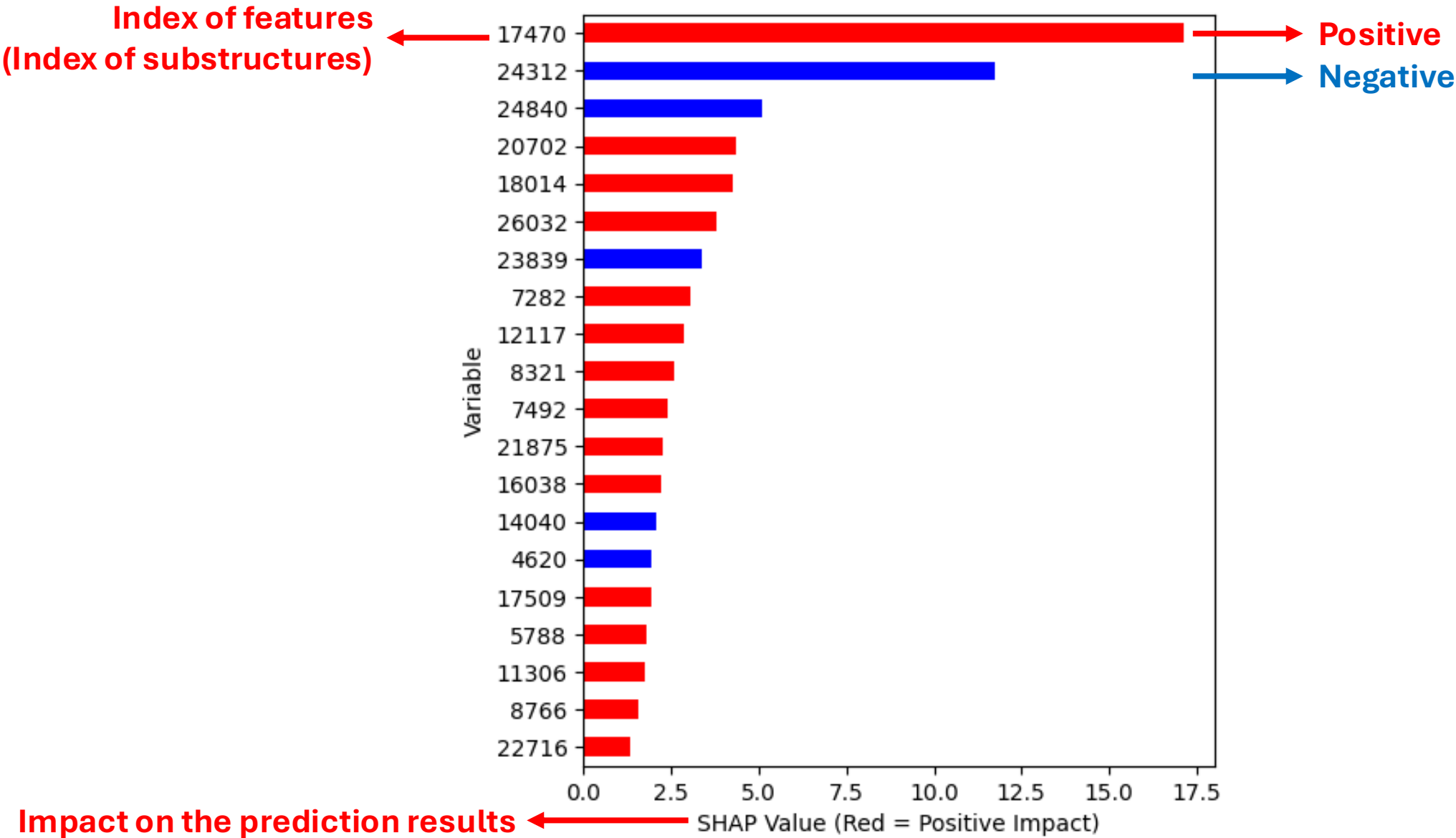
Get SHAP analysis results

Get Morgan Fingerprint with Frequency information

Evaluate the impact of each feature index
(substructure) on the prediction results

Plot the result with a bar chart

SHAP (SHapley Additive exPlanations) analysis



SHAP (SHapley Additive exPlanations) analysis

```
tpls = []
PlotSub = []
legs = []

for i in range(len(Tg_df[100:])):
    mol = Chem.MolFromSmiles(Tg_df.Smiles.iloc[i])
    bi = {}
    fp = rdMolDescriptors.GetMorganFingerprint(mol, radius=3, bitInfo=bi)
    fp_n = fp.GetNonzeroElements()
    for x in k2.iloc[:, -1]['Variable'][:12]:
        if x not in PlotSub:
            Code = (Corr_df[Corr_df['index'] == x][0].values[0])
            if Code in fp_n.keys():
                print(x)
                legs.append(x)
                tpls.append((mol, Code, bi))
                img = Draw.DrawMorganBit(mol, Code, bi)
                PlotSub.append(x)

img = Draw.DrawMorganBits(tpls, molsPerRow=4, legends=[str(x) for x in legs])
img
```

→ The loop range in the dataset (determined by the dataset's size)

→ The loop range in the SHAP analysis results (top X most important substructures)

→ Print the index

} → Identify the substructures

→ Plot the substructures

24312
23839
17470
18014
7282
24840
26032
8321
20702
12117
21875
7492

→ Output

SHAP (SHapley Additive exPlanations) analysis

```
tpls = []
PlotSub = []
legs = []

for i in range(len(Tg_df[100:])):
    mol = Chem.MolFromSmiles(Tg_df.Smiles.iloc[i])
    bi = {}
    fp = rdMolDescriptors.GetMorganFingerprint(mol, radius=3, bitInfo=bi)
    fp_n = fp.GetNonzeroElements()
    for x in k2.iloc[0:12]['Variable']:
        if x not in PlotSub:
            Code = (Corr_df[Corr_df['index'] == x][0].values[0])
            if Code in fp_n.keys():
                print(x)
                legs.append(x)
                tpls.append((mol, Code, bi))
                img = Draw.DrawMorganBit(mol, Code, bi)
                PlotSub.append(x)

img = Draw.DrawMorganBits(tpls, molsPerRow=4, legends=[str(x) for x in legs])
img
```

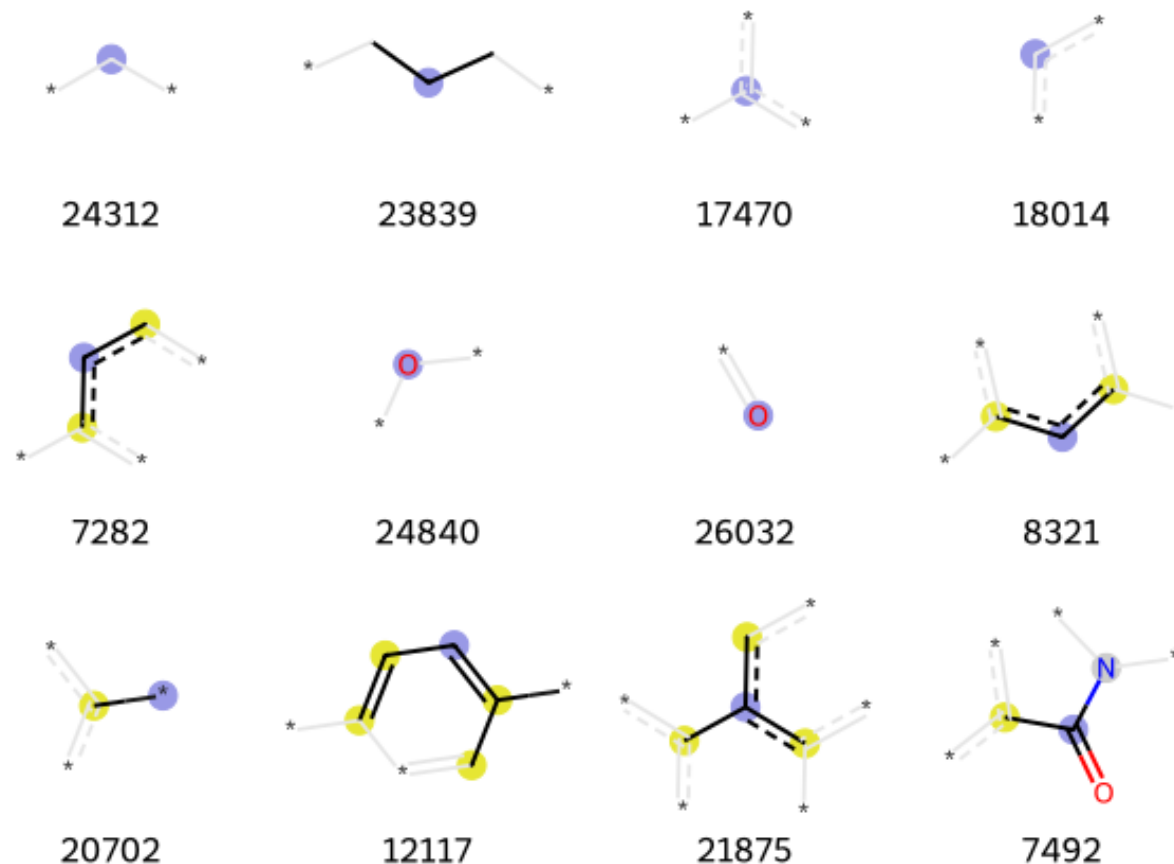
→ The loop range in the dataset (determined by the dataset's size)

→ The loop range in the SHAP analysis results (top X most important substructures)

→ Print the index

24312
23839
17470
18014
7282
24840
26032
8321
20702
12117
21875
7492

→ Output

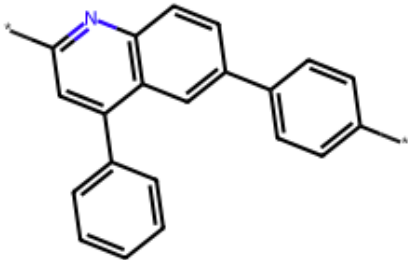


SHAP (SHapley Additive exPlanations) analysis

```
index_polymer = 66  
  
mol = Chem.MolFromSmiles(Tg_df_sampled.loc[index_polymer, 'Smiles'])  
mol
```

The index of the polymer we are interested in

Plot the polymer molecular structure based on its SMILES

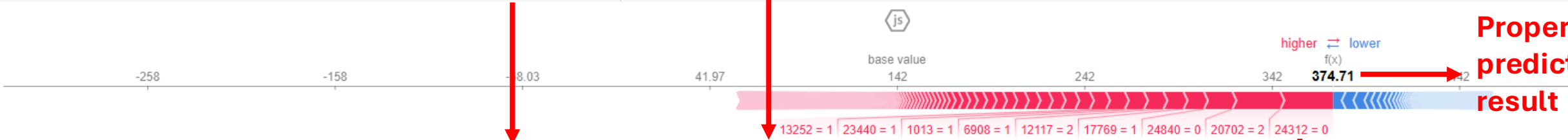


```
explainer = shap.KernelExplainer(model.predict,X.values)  
  
216/216 [=====] - 1s 5ms/step  
  
shap.initjs()  
shap.force_plot(explainer.expected_value[0], shap_values[0][index_polymer,:],MFF_sampled.values[index_polymer,:],feature_names=list(MFF_sampled.columns))
```

Run the explainer

Morgan Fingerprint with Frequency information

Plot the result

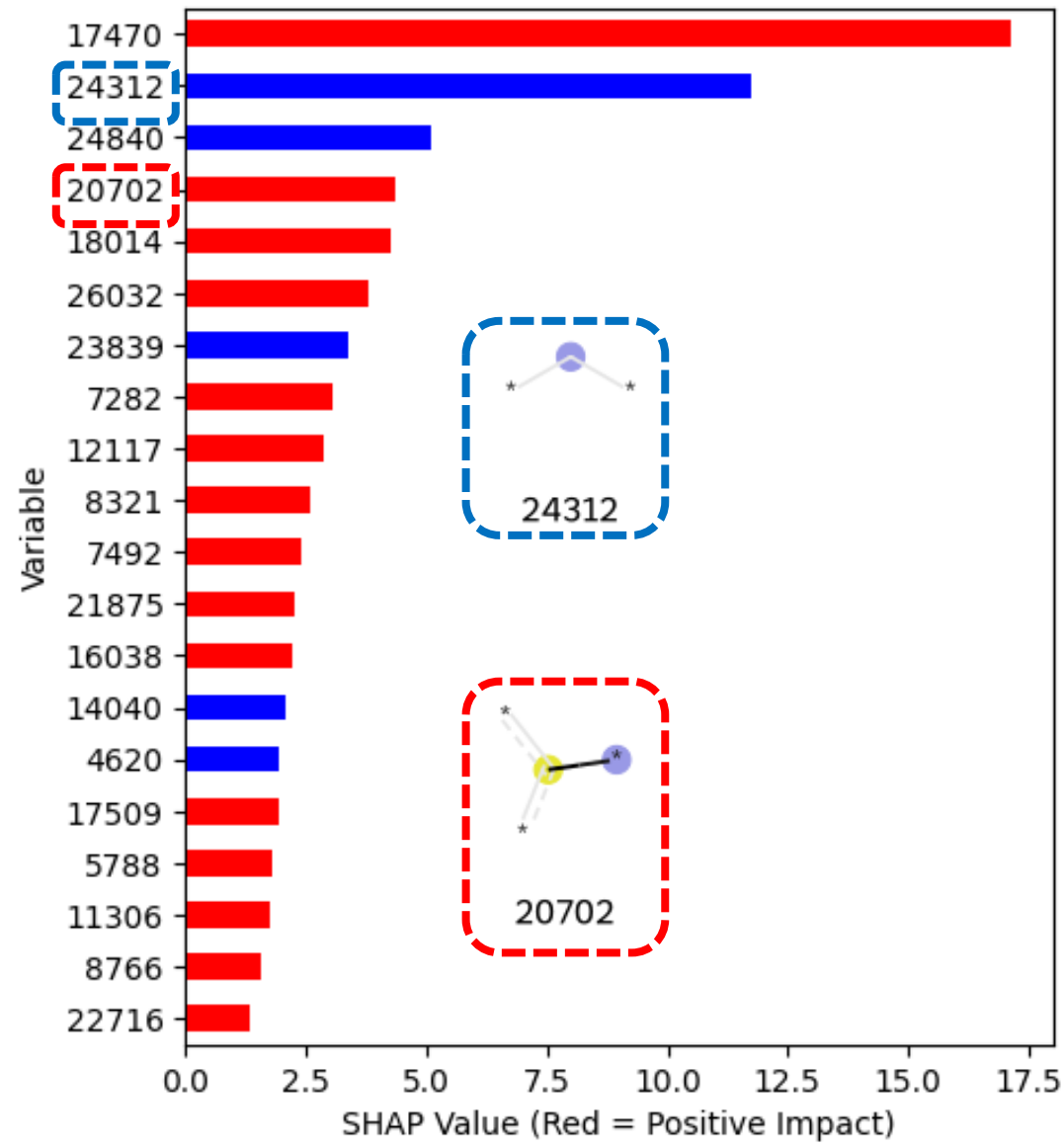


The index of the polymer we are interested in

Feature index (substructure)

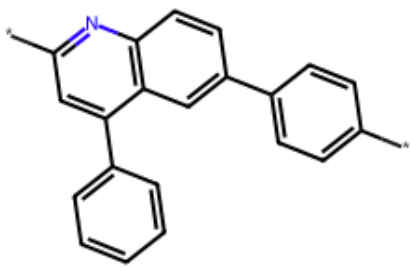
SHAP (SHapley Additive exPlanations) analysis

```
index_polymer = 66
```



That is

```
index_polymer = 66  
  
mol = Chem.MolFromSmiles(Tg_df_sampled.loc[index_polymer, 'Smiles'])  
mol
```

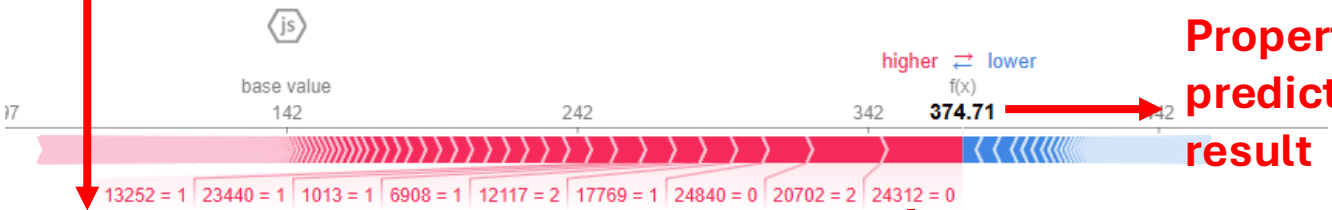


Morgan Fingerprint with Frequency information

the explainer

```
shap_values[index_polymer, :], feature_names=list(MFF_sampled.columns))
```

Plot the result



Interested in

Property prediction result

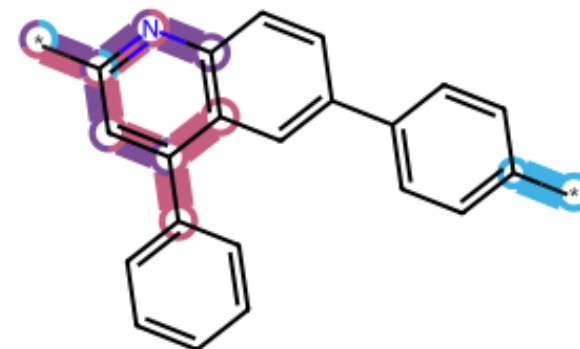
Feature index (substructure)

SHAP (SHapley Additive exPlanations) analysis

```
def countdown(centric, radius, mol, Highlight_Atom, Highlight_Bond):  
#   centric: central atom ID  
#   radius: serch radius  
#   mol:     molecules  
#   Highlight_Atom: All atoms within radius of a central atom  
  
if radius < 0:  
    return  
else:  
    Highlight_Atom.append(centric)  
    atom = mol.GetAtomWithIdx(centric)  
    for nei in atom.GetNeighbors():  
        if radius > 0:  
            Highlight_Bond.append(mol.GetBondBetweenAtoms(centric, nei.GetIdx()).GetIdx())  
            countdown(nei.GetIdx(), radius - 1, mol, Highlight_Atom, Highlight_Bond)
```

```
COLS = [(0.0, 0.7, 0.9), (0.8, 0.35, 0.5), (0.5, 0.3, 0.6), (0.6, 0.8, 0.0)]
```

Define the colors used to identify substructures



The index of the polymer we are interested in

```
smi = Tg_df_sampled.loc[66, 'Smiles']  
substructures = [20702, 6908, 17769]  
do_a_picture(smi, substructures, '/content/drive/My Drive/SPE/Highest_Modulus_substructure.png', '', fmt='png')
```

The index of the most important substructures

The path where the image is saved

Image save format

SHAP (SHapley Additive exPlanations) analysis

```
col = 'Tg(C)'
with open("shap_values_{}.pkl".format(col), "wb") as f:
    pickle.dump(shap_values, f)

with open("shap_values_{}.pkl".format(col), "rb") as f:
    shap_values = pickle.load(f)
```

Record the SHAP information for later PCA analysis (Tg)

```
col = 'Eg(eV)'
with open("shap_values_{}.pkl".format(col), "wb") as f:
    pickle.dump(shap_values, f)

with open("shap_values_{}.pkl".format(col), "rb") as f:
    shap_values = pickle.load(f)
```

Record the SHAP information for later PCA analysis (Eg)

```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
```

Import PCA from the scikit-learn package

```
shap_df_Tg = pickle.load(open("shap_df_Tg(C).pkl", "rb"))
shap_df_Eg = pickle.load(open("shap_df_Eg(eV).pkl", "rb"))
shap_All = shap_df_Tg.merge(shap_df_Eg, on = 'Variable')
shap_All['SHAP_abs_x'] = shap_All['SHAP_abs_x']/max(shap_All['SHAP_abs_x'])*shap_All['Corr_x']/abs(shap_All['Corr_x'])
shap_All['SHAP_abs_y'] = shap_All['SHAP_abs_y']/max(shap_All['SHAP_abs_y'])*shap_All['Corr_y']/abs(shap_All['Corr_y'])
rows_with_nan = shap_All[shap_All.isna().any(axis=1)]
shap_All=shap_All.dropna()
pca = PCA(n_components=2)
pca_array = pca.fit_transform(shap_All[['SHAP_abs_x', 'SHAP_abs_y']])

# Convert back to DataFrame for readability
df_pca = pd.DataFrame(data=pca_array)
df_pca.columns = ['PC' + str(col+1) for col in df_pca.columns.values]
pc1 = pca.components_[0]
pc2 = pca.components_[1]
```

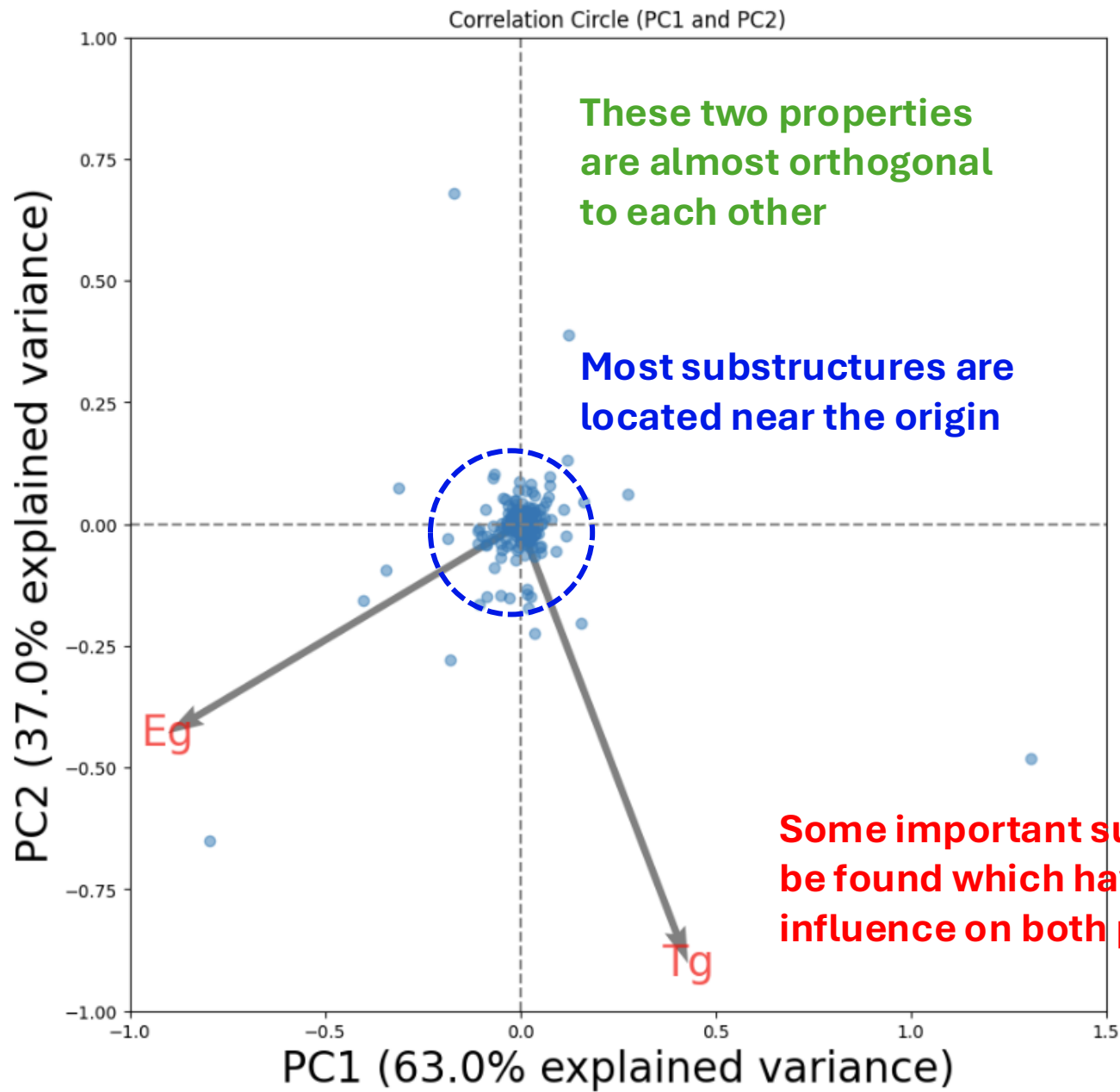
Load the SHAP information

Perform PCA analysis

SHAP (SHapley Additive exPlanations) analysis

```
pcs = pca.components_  
display_circles(pcs, 2, pca, [(0,1)], labels = np.array(['Tg', 'Eg'],),)
```

Plot the PCA analysis result



Check the PC1 and PC2 values of each substructure

```
shap_All_reset = shap_All.reset_index(drop=True)  
df_pca['substructures'] = shap_All_reset['Variable']  
df_pca
```

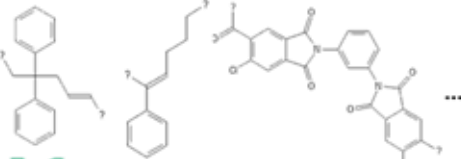
	PC1	PC2	substructures
0	1.308097	-0.481462	17470
1	-0.169930	0.679423	24312
2	0.122568	0.389560	24840
3	0.154365	-0.204157	20702
4	-0.796770	-0.650494	18014
...	...	...	...
919	-0.003986	0.000962	11793
920	-0.003657	0.001543	4779
921	0.001989	0.004232	18881
922	-0.000149	0.003183	11108
923	-0.000633	0.002635	21292

924 rows x 3 columns

Data shortage

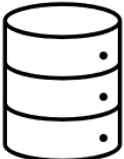


Real Polymer Structures

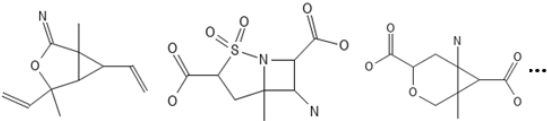


 ~18,000 Real Polymers

V.S.



Small Molecule Compounds



 ~116M Compounds



Property

Count

Log-transformed octanol/water partition coefficient ($\log P$)

14,207

Blood-brain barrier (BBB) permeability

7,807



Property

Count

Glass transition temperature

6,906

Melting temperature

3,633

Decomposition temperature

5,237

Thermal conductivity

68

Heat capacity

79

Property

Count

Young's modulus

923

Yield strength

230

Break strength

982

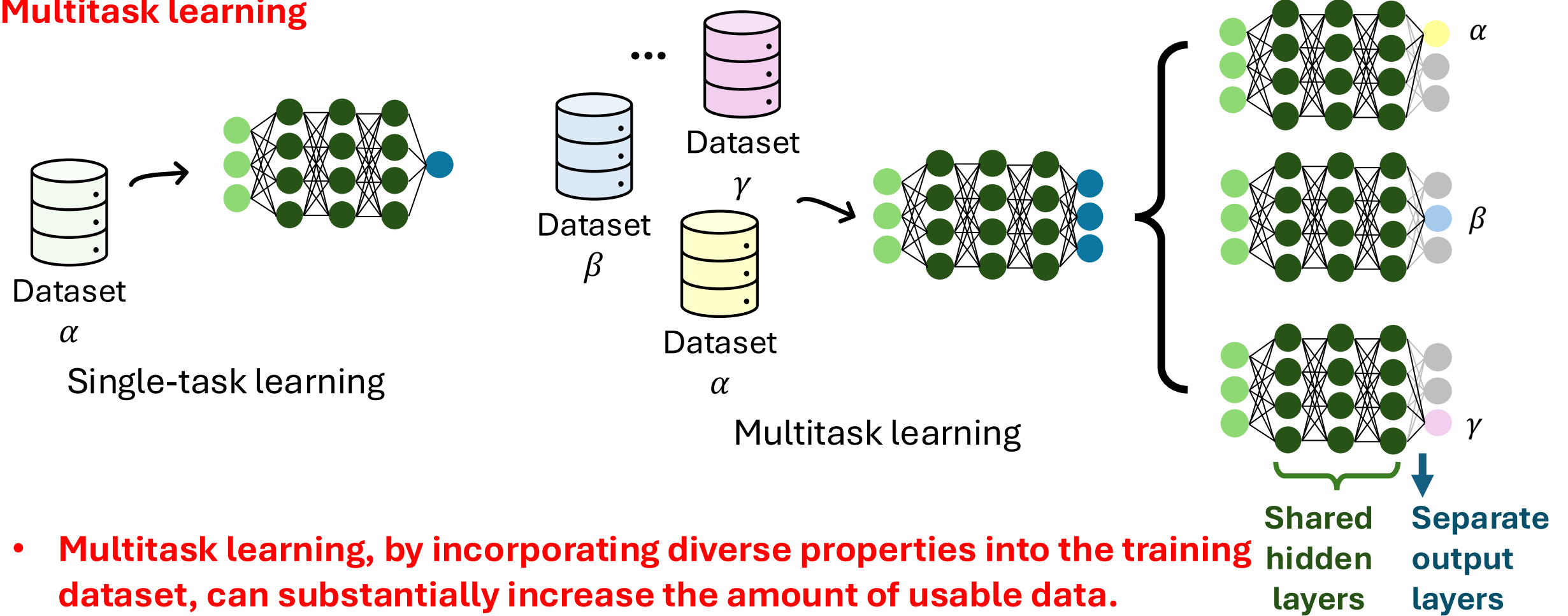
Dielectric constant

627

Bandgap

3,379

Multitask learning



Training Prediction Models with Limited Data

Multitask learning

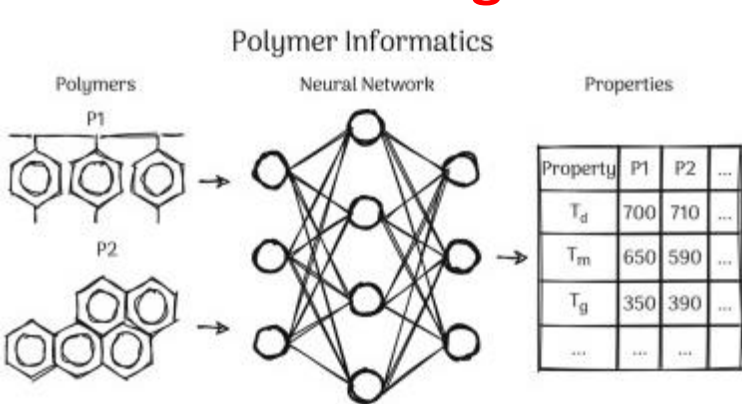
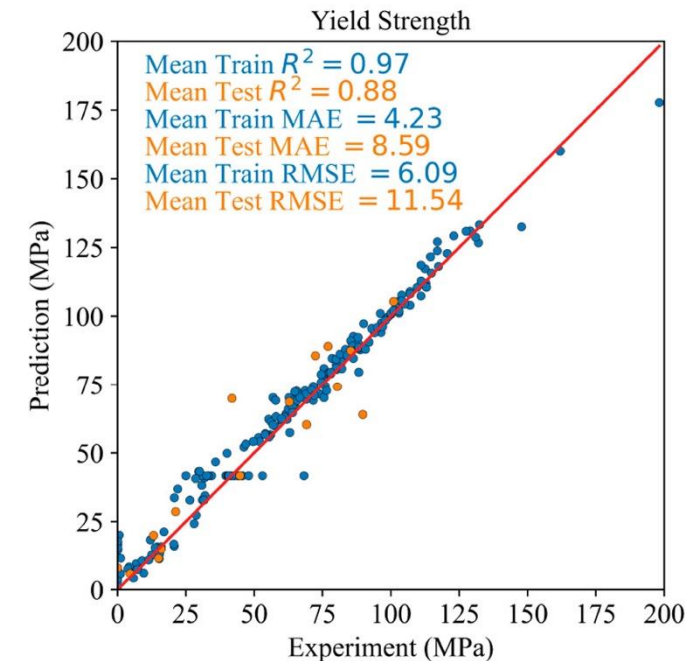
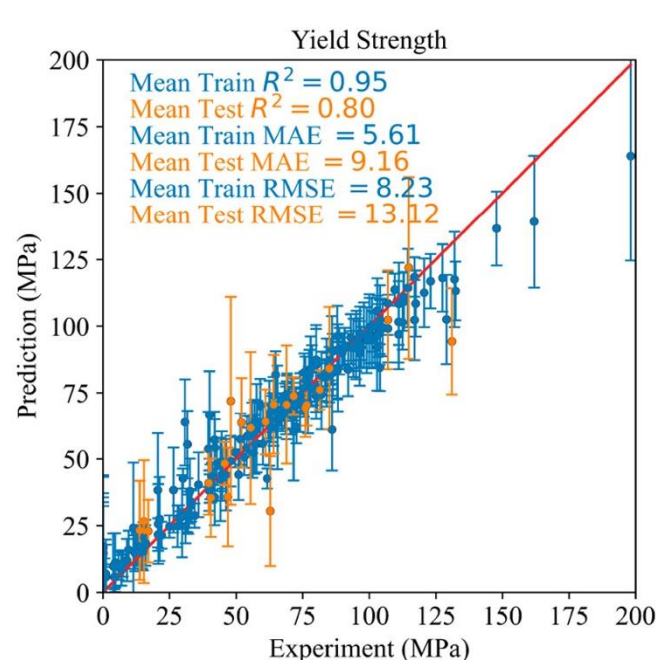
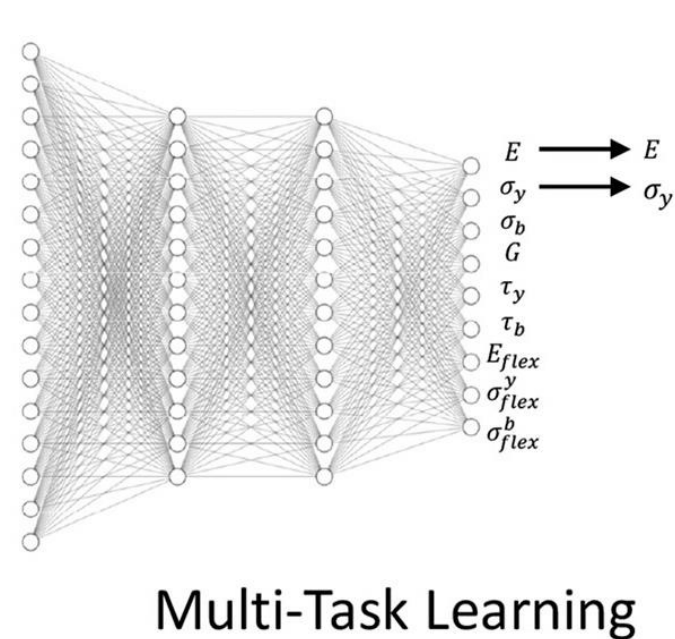


Table 2. Averages of the normalized root mean squared errors values

Model	All	Categories					
		Thermal	Thermod. & physical	Electronic	Optical & dielectric	Mech.	Solubility & permeability
GP-ST	0.93	0.92	0.86	0.88	0.98	1.00	0.93
NN-ST	0.89	0.95	0.76	0.91	0.91	0.95	0.94
NN-MT1	0.89	0.98	0.83	0.97	0.83	0.94	0.92
NN-MT2-all	0.79	0.97	0.82	0.96	0.69	0.89	0.70
NN-MT2-sub	0.65	0.94	0.87	0.79	0.47	0.82	0.46

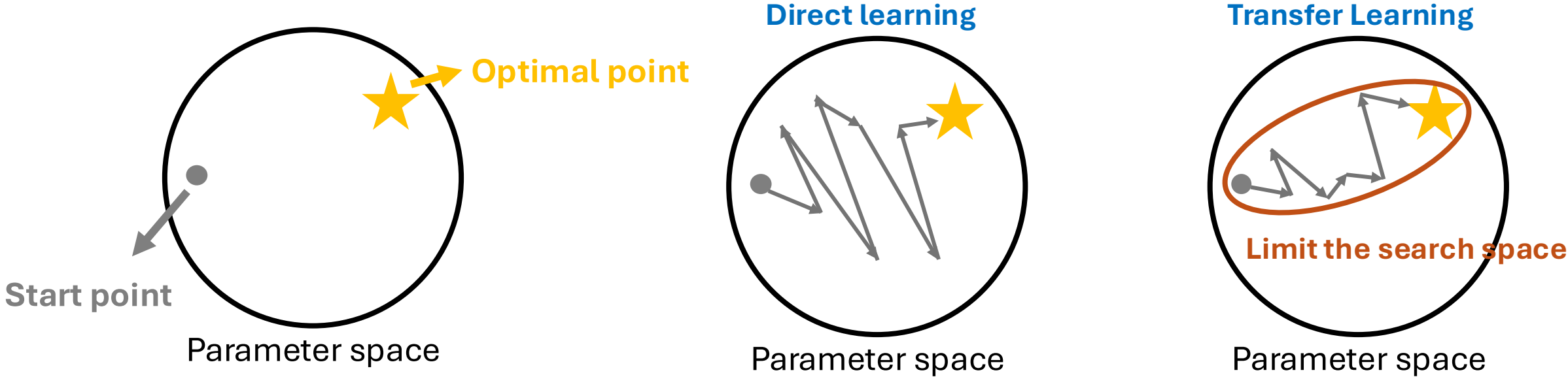
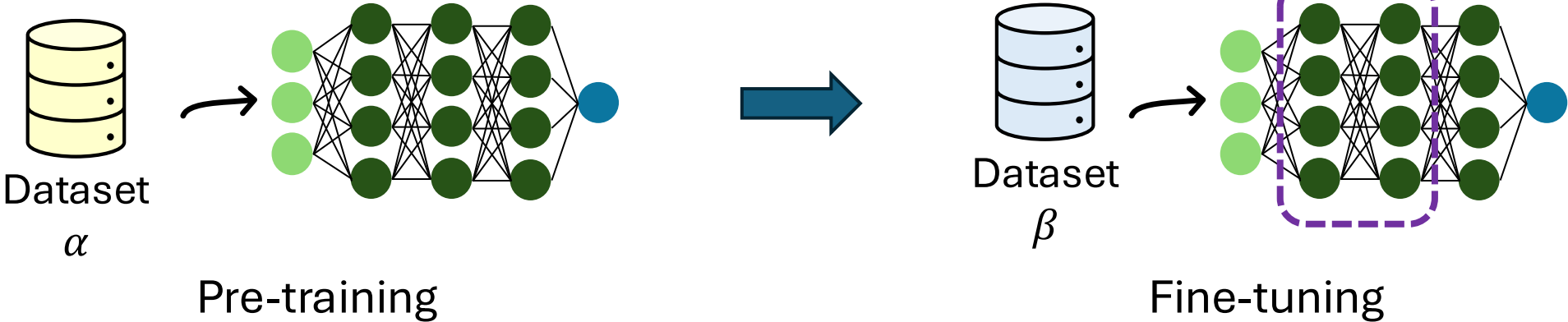
The root mean squared error averages were normalized for each property so that the maximal value is 1.

Kuenneth, C., Rajan, A.C., Tran, H., Chen, L., Kim, C. and Ramprasad, R., 2021. Polymer informatics with multi-task learning. *Patterns*, 2(4).



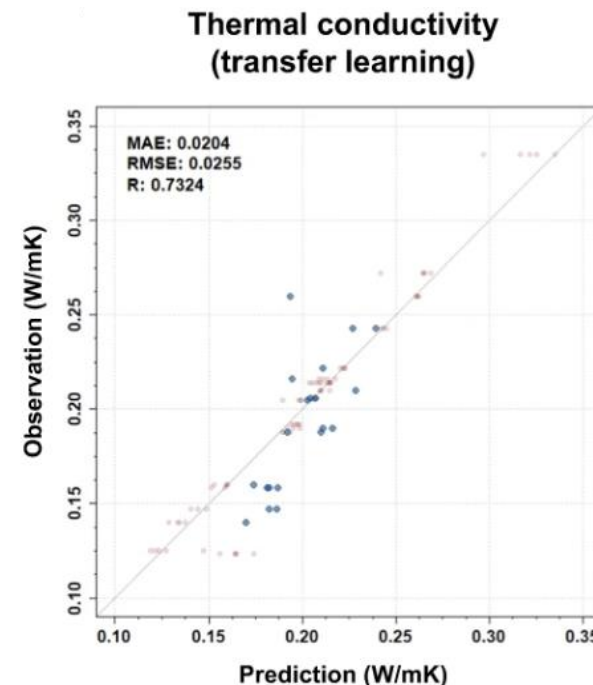
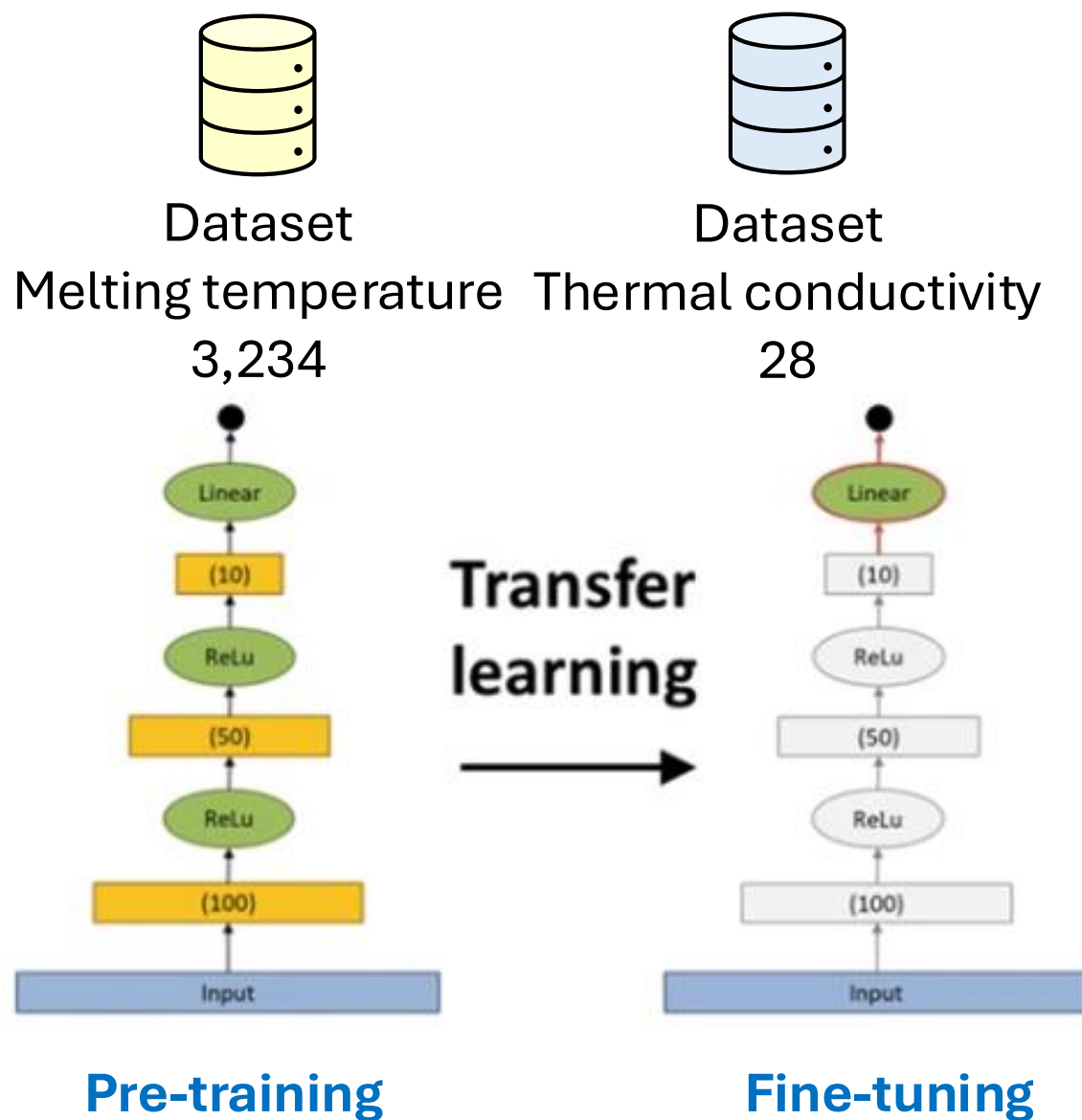
Yue, T., He, J., Tao, L. and Li, Y., 2023. High-throughput screening and prediction of high modulus of resilience polymers using explainable machine learning. *Journal of Chemical Theory and Computation*, 19(14), pp.4641-4653.

Transfer Learning

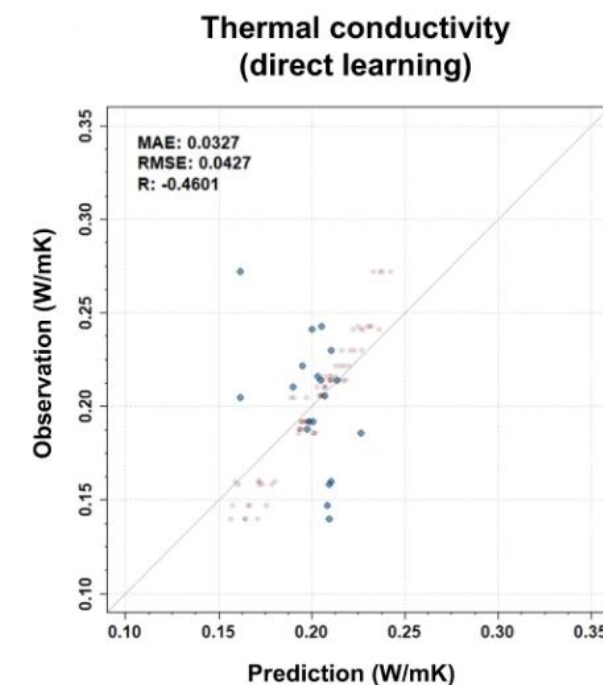


Training Prediction Models with Limited Data

Transfer Learning



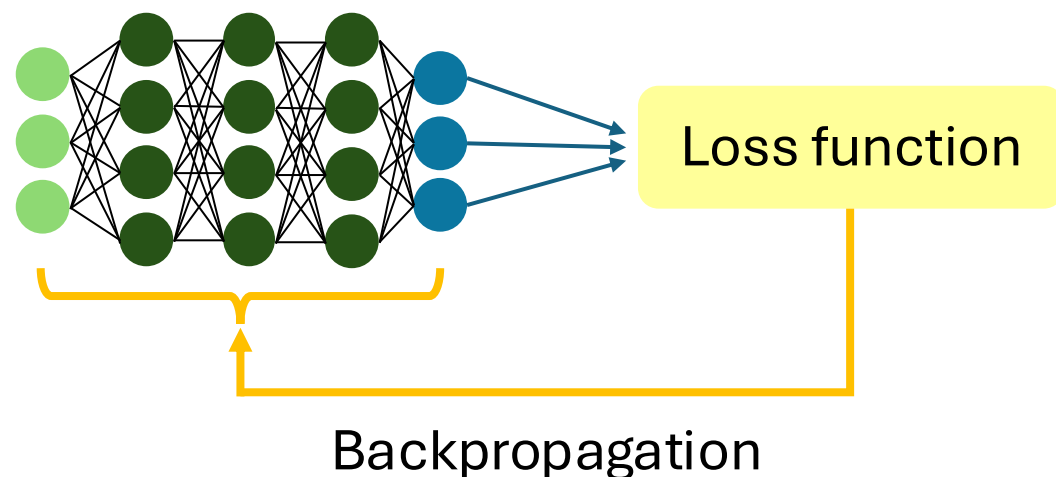
$$R^2: 0.73$$



$$R^2: -0.46$$

Wu, S., Kondo, Y., Kakimoto, M.A., Yang, B., Yamada, H., Kuwajima, I., Lambard, G., Hongo, K., Xu, Y., Shiomi, J. and Schick, C., 2019. Machine-learning-assisted discovery of polymers with high thermal conductivity using a molecular design algorithm. *Npj Computational Materials*, 5(1), p.66.

Physics-informed Neural Networks



Traditional NN: Data loss

$$\mathcal{L} = (A - \hat{A})^2 + (B - \hat{B})^2 + (C - \hat{C})^2$$

PINN: Data loss + Physics loss

$$\mathcal{L}_1 = (A - \hat{A})^2 + (B - \hat{B})^2 + (C - \hat{C})^2$$

$$\mathcal{L}_2 = f(\hat{A}, \hat{B}, \hat{C})$$

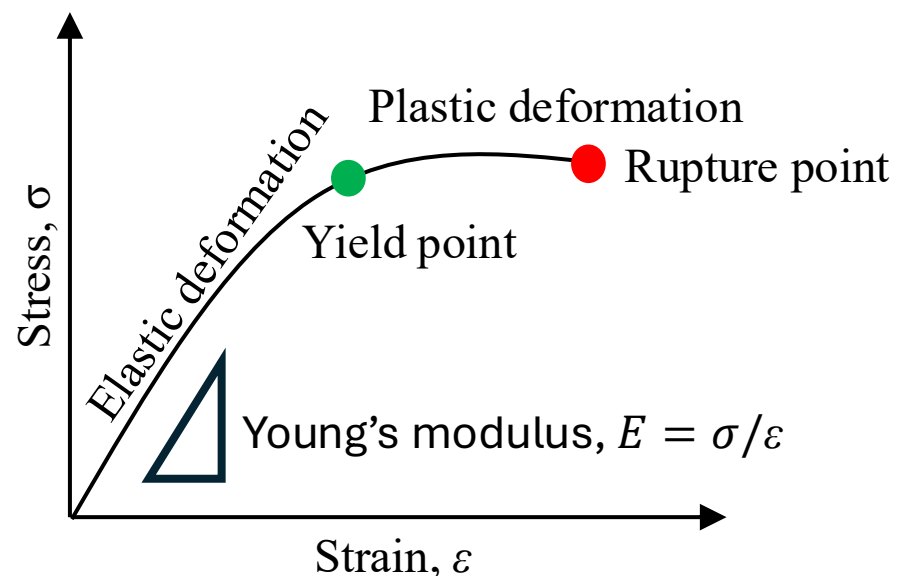
$$\mathcal{L} = \mathcal{L}_1 + \lambda \mathcal{L}_2$$

Only based on prediction values

A, B, C label values

$\hat{A}, \hat{B}, \hat{C}$ predicted values

$f(\cdot)$ enforce equations/ governing equations



$$\mathcal{L}_1 = (\hat{E} - E)^2 + (\hat{\sigma} - \sigma)^2 + (\hat{\varepsilon} - \varepsilon)^2$$

$$\mathcal{L}_2 = \hat{E} - \hat{\sigma}/\hat{\varepsilon}$$

$$\mathcal{L} = \mathcal{L}_1 + \lambda \mathcal{L}_2$$

E, σ, ε label values

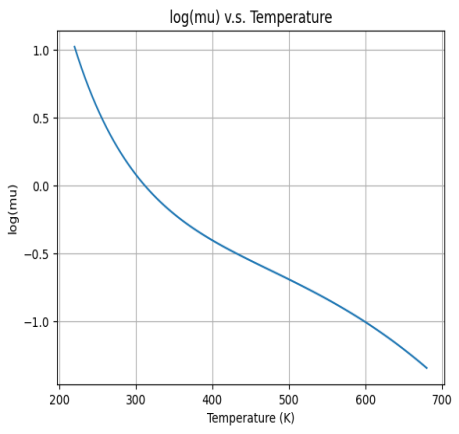
$\hat{E}, \hat{\sigma}, \hat{\varepsilon}$ predicted values

Training Prediction Models with Limited Data

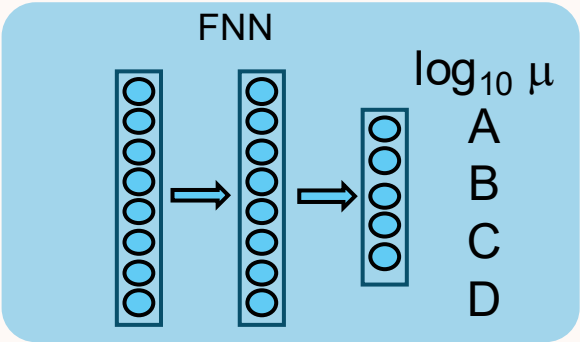
Physics-informed Neural Networks

Transport Properties of Chemicals and Hydrocarbons

$$\log_{10} \mu = A + B/T + CT + DT^2$$



6,120 unique compounds



Loss function

$$\mathcal{L}_1 = (A - \hat{A})^2 + (B - \hat{B})^2 + (C - \hat{C})^2 + (D - \hat{D})^2$$

$$\mathcal{L}_2 = [\log(\mu)' - \log(\mu)]^2$$

$$\mathcal{L}_3 = \left[\log_{10} \mu - \left(\hat{A} + \frac{\hat{B}}{T} + \hat{C}T + \hat{D}T^2 \right) \right]^2$$

$$\mathcal{L} = \mathcal{L}_1 + \mathcal{L}_2 + \lambda \mathcal{L}_3$$

Data loss

Physics loss

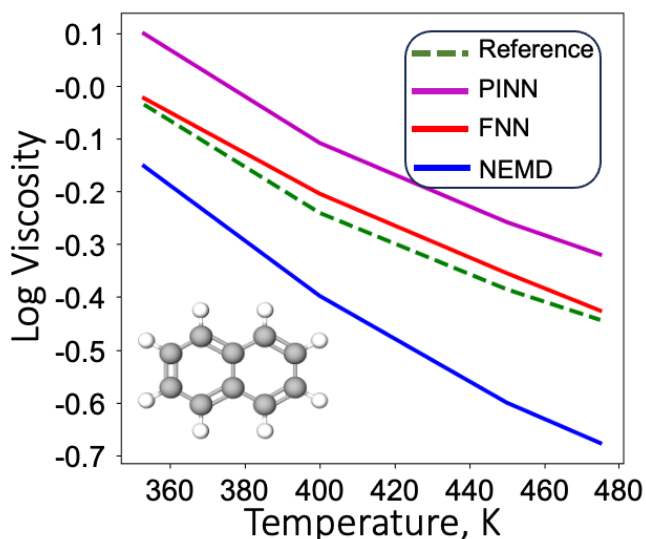
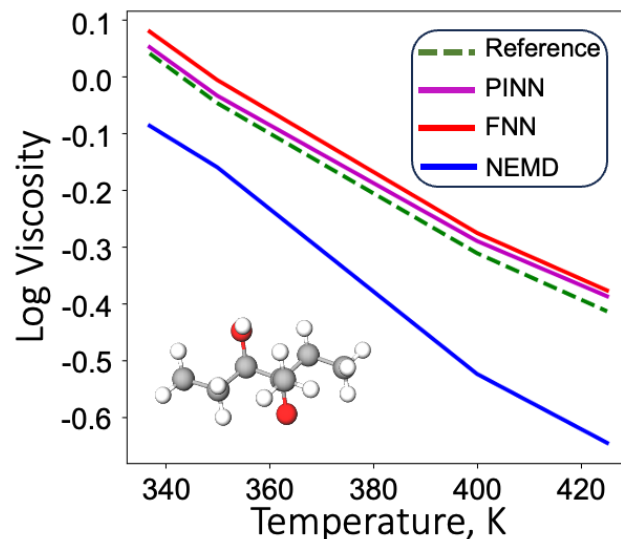
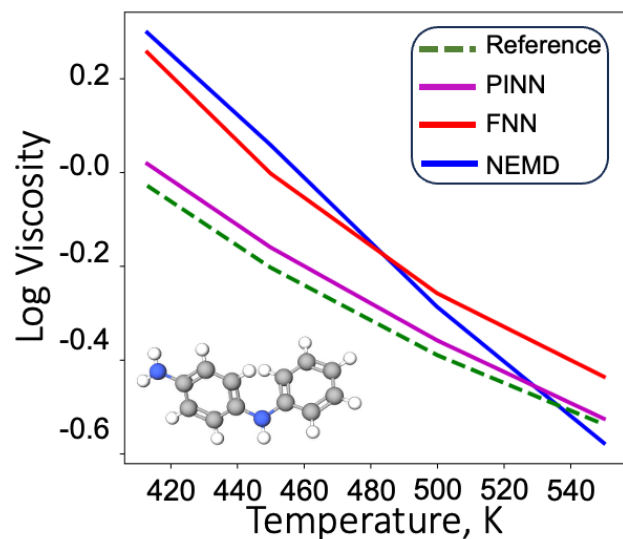
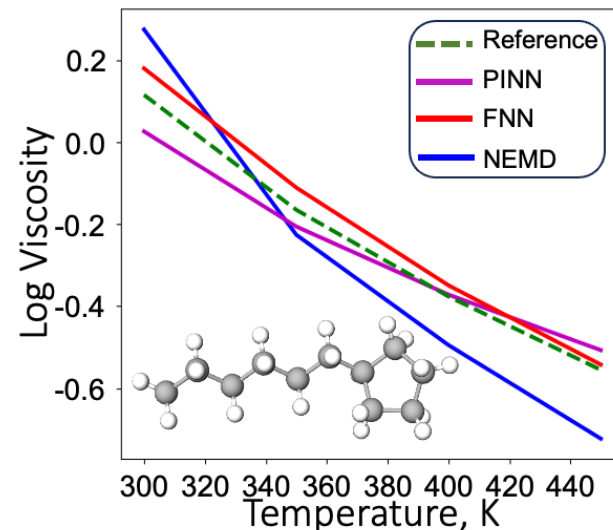
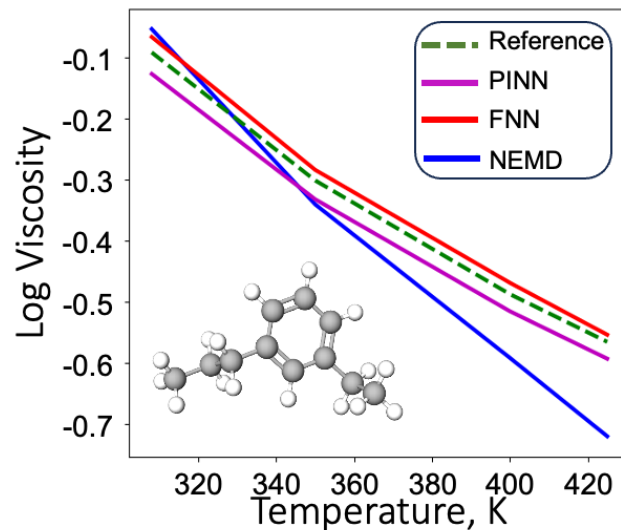
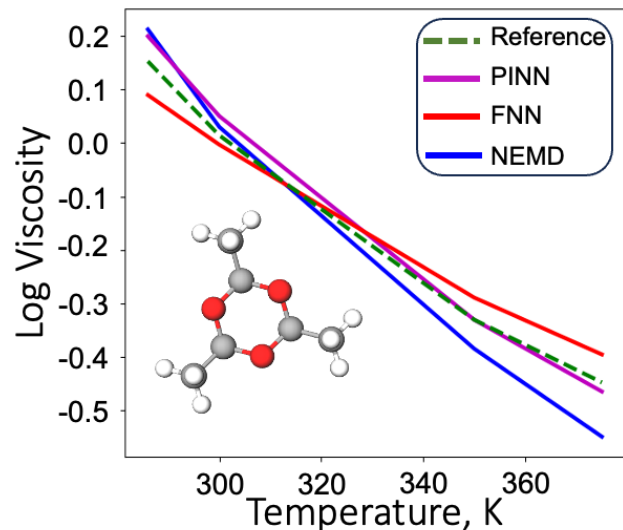
λ	R^2 (pred log μ)	R^2 (cal log μ)	R^2 (pred A)	R^2 (pred B)	R^2 (pred C)	R^2 (pred D)
0.00001	0.99	0.91	0.97	0.98	0.97	0.97
0.001	0.98	0.90	0.95	0.96	0.95	0.95
0.01	0.98	0.91	0.95	0.96	0.95	0.95
0.1	0.99	0.97	0.96	0.96	0.96	0.95
1	0.96	0.95	0.77	0.84	0.71	0.68
-	-	0.63	0.75	0.82	0.67	0.71

Only data loss

(@330k)

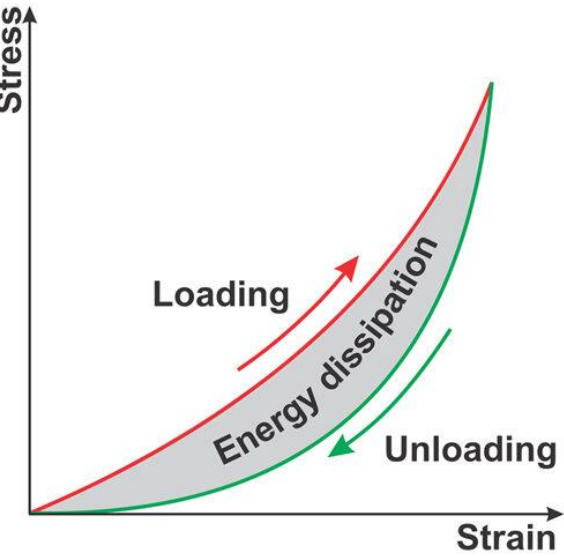
Training Prediction Models with Limited Data

Physics-informed Neural Networks



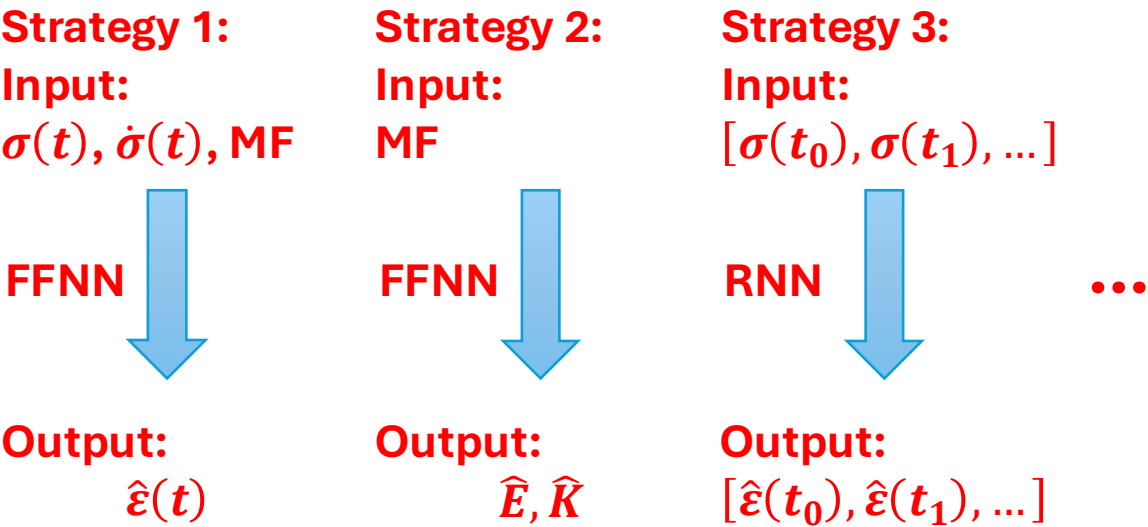
- **Temperature-dependency can be captured so well.**
- **Physics loss also contribute to parameter optimization.**

Quick Example: Physics-informed neural networks and Stress-Strain Curves Prediction



$$\varepsilon(t) = \frac{\sigma(t)}{E} + \int_0^t K(t - t')\dot{\sigma}(t')dt'$$

where
 t is time,
 ε is strain,
 σ is stress,
 E is elastic modulus for creep,
 K is creep function



```
# Define the input layer with a shape of 1024 features
A1 = Input(shape=[1024],name='A1')
# Define the hidden layers
A2 = Dense(n_1, activation='relu',name='A2')(A1)
A3 = Dense(n_2, activation='relu',name='A3')(A2)
A4 = Dense(n_3, activation='relu',name='A4')(A3)
A5 = Dense(n_4, activation='relu',name='A5')(A4)
A6 = Dense(1, name='A6')(A5)
# Compile the model with Mean Squared Error (MSE) loss function and Adam optimizer
model = Model(inputs=[A1],outputs=[A6])
model.compile(loss = "mse", optimizer = 'adam')
```

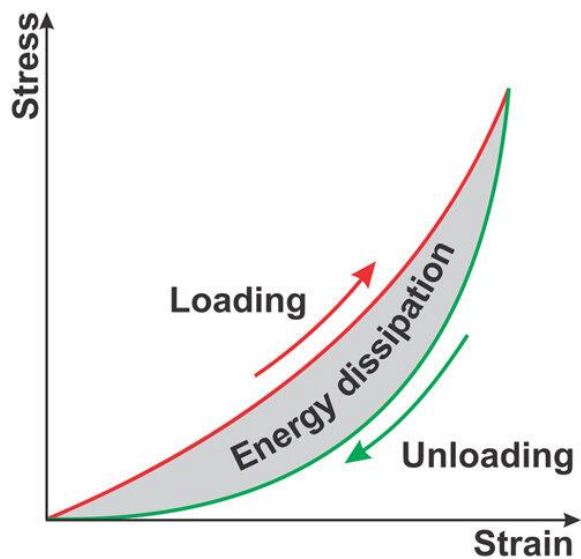
Strategy 1:

$$\min_p \frac{1}{n} \sum_{i=1}^n (\hat{\varepsilon}(t) - \varepsilon(t))^2$$

Strategy 2:

$$\min_p \left[\frac{1}{n} \sum_{i=1}^n (\hat{E} - E)^2 + \frac{1}{n} \sum_{i=1}^n (\hat{K} - K)^2 \right]$$

Quick Example: Physics-informed neural networks and Stress-Strain Curves Prediction



$$\varepsilon(t) = \frac{\sigma(t)}{E} + \int_0^t K(t - t') \dot{\sigma}(t') dt'$$

Where
 t is time,
 ε is strain,
 σ is stress,
 E is elastic modulus for creep,
 K is creep function

Strategy Physics-informed neural networks :

Input:

$$\sigma(t), \dot{\sigma}(t), \text{MF}$$

FFNN



Output:

$$\hat{\varepsilon}(t), \hat{E}, \hat{K}$$

Loss function:

$$\mathcal{L} = \mathcal{L}_1 + \lambda \mathcal{L}_2$$

where

$$\mathcal{L}_1 = (\hat{\varepsilon}(t) - \varepsilon(t))^2 + (\hat{E} - E)^2 + (\hat{K} - K)^2$$

$$\mathcal{L}_2 = \left(\hat{\varepsilon}(t) - \frac{\sigma(t)}{\hat{E}} + \int_0^t \hat{K}(t - t') \dot{\sigma}(t') dt' \right)^2$$

Quick Example: Physics-informed neural networks and Stress-Strain Curves Prediction

Directly feed the input into the output layer to compute the error

```
stress = Input(shape=(1,), name='stress')
dstress = Input(shape=(1,), name='dstress')

combined = Concatenate()([A6, stress, dstress])

model = Model(inputs=[A1, stress, dstress], outputs=combined)

# Compile the model with the custom loss function
model.compile(loss=PINNLoss(), optimizer=Adam())
```

Use a custom PINN loss function

```
class PINNLoss(tf.keras.losses.Loss):
    def __init__(self, name="PINN_loss", **kwargs):
        super().__init__(name=name, **kwargs)

    def call(self, y_true, y_pred):
        pstrain, pE, pK, stress, dstress = tf.split(y_pred, [1, 1, 1, 1, 1, 1], axis=1)
        strain, E, K = tf.split(y_true, [1, 1, 1], axis=1)

        dt = 0.01

        loss_1 = tf.square(pstrain - strain)
        loss_2 = tf.square(pE - pK)
        loss_3 = tf.square(pK - K)
        loss_4 = tf.square(pstrain - stress / pE - tf.reduce_sum(pK * dstress * dt))

        total_loss = loss_1 + loss_2 + loss_3 + 0.1*(loss_4)

        return total_loss
```

Get predicted results

Get true values

Weight λ

Strategy Physics-informed neural networks :
Input:

$$\sigma(t), \dot{\sigma}(t), MF$$

FFNN

Output:

$$\hat{\varepsilon}(t), \hat{E}, \hat{K}$$

Loss function:

$$\mathcal{L} = \mathcal{L}_1 + \lambda \mathcal{L}_2$$

where

$$\mathcal{L}_1 = (\hat{\varepsilon}(t) - \varepsilon(t))^2 + (\hat{E} - E)^2 + (\hat{K} - K)^2$$

$$\mathcal{L}_2 = \left(\hat{\varepsilon}(t) - \frac{\sigma(t)}{\hat{E}} + \int_0^t \hat{K}(t - t') \dot{\sigma}(t') dt' \right)$$

Question Time

Thank you!

Tianle Yue (tyue4@wisc.edu)